ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

KIẾN TRÚC PHẦN MỀM (CO3017)

BÀI TẬP LỚN

INTELLIGENT RESTAURANT MANAGEMENT SYSTEM

Giảng viên hướng dẫn: Trần Trương Tuấn Phát
Lớp: L01
Sinh viên thực hiện: Lữ Quốc Pháp - 2312562
Nguyễn Lê Nguyên - 2312365
Trần Trọng Nghĩa - 2312285
Lê Sĩ Nhật Quang - 2312791
Nguyễn Đức Phương - 2312749

THÀNH PHỐ HỒ CHÍ MINH, 2026

Mục lục

Danh mục hình vẽ **3**

Danh mục bảng biểu **3**

Danh sách thành viên và công việc **4**

1 Bối cảnh hệ thống Intelligent Restaurant Management System **5**

1.1 Giới thiệu và mục tiêu 5

1.2 Phạm vi hệ thống và các phân hệ chức năng 5

1.2.1 Đặt món số và quản lý thực đơn 5

1.2.2 Hiển thị đơn bếp và điều phối luồng chế biến 6

1.2.3 Quản lý bàn và đặt chỗ 6

1.2.4 Tính tiền, thanh toán và xuất hóa đơn 7

1.2.5 Giám sát tồn kho 7

1.2.6 Phân tích và báo cáo 7

1.2.7 Công cụ quản trị 8

1.3 Tác nhân và tương tác vận hành điển hình 8

1.4 Yêu cầu chức năng 9

1.5 Yêu cầu phi chức năng 11

1.6 Ràng buộc và giả định 11

1.7 Architecture Characteristics 12

1.8 Tiêu chí thành công 12

2 Use Case Scenarios - System Modeling **14**

3 So sánh và lựa chọn phong cách kiến trúc phù hợp **35**

3.1 Ước tính tải hệ thống 35

3.2 Modular Monolith 35

3.3 Microservices 36

3.4 Event-Driven 36

3.5 Kết luận lựa chọn: Modular Monolith kết hợp Event-Driven có chọn lọc . 36

3.5.1 Ranh giới Modular Monolith 37
3.5.2 Phạm vi áp dụng Event-Driven 37
3.5.3 Lý do không chọn Microservices ngay từ đầu 38

4 Xây dựng kiến trúc phần mềm cho hệ thống 39

4.1 Module Views 39
4.1.1 Context View 39
4.1.2 Package View 41
4.1.3 Module View 43
4.2 Component & Connector Views 45
4.2.1 Core Business Flow Component View 45
4.2.2 Event-Driven Component View 49
4.2.3 Persistence Component View 52
4.2.4 Infrastructure / Technical Component View 54
4.2.5 Component & Connector (C&C) View 57
4.3 Allocation View 60
4.3.1 Install Style 61
4.3.2 Deployment Style 63
4.3.3 Work Assignment Style 64

5 Thiết kế class diagram 66

5.1 Nghiệp vụ 1: Nhân viên thực hiện chọn món và tạo order 66
5.2 Nghiệp vụ 2: Quản trị viên thiết lập Catalog (Menu và Khuyến mãi) . . 68
5.3 Nghiệp vụ 3: Xử lý nhà bếp và Tự động trừ tồn kho 70
5.4 Nghiệp vụ 4: Thanh toán hóa đơn và Quản lý trạng thái bàn 72
5.5 Nghiệp vụ 5: Phân tích dữ liệu và Xuất báo cáo 74
5.6 Nghiệp vụ 6: Kiểm soát truy cập và Lưu vết hệ thống (Cross-Cutting) . 76

6 Link mã nguồn 77

Danh mục hình vẽ

1 Context View của hệ thống 39
2 Package View của hệ thống 41
3 Module View của hệ thống 43
4 Core Business Flow Component View 45
5 Event-Driven Component View 49
6 Persistence Component View 52
7 Infrastructure / Technical Component View 54
8 Component & Connector View của hệ thống 57
9 Install Style của hệ thống 61
10 Deployment Style của hệ thống 63
11 Thiết kế class diagram cho nghiệp vụ tạo order 66
12 Thiết kế class diagram cho nghiệp vụ quản lý danh mục và khuyến mãi . 68
13 Thiết kế class diagram cho nghiệp vụ chế biến và quản lý nguyên liệu . . 70
14 Thiết kế class diagram cho nghiệp vụ thanh toán và giải phóng bàn . . . 72
15 Thiết kế class diagram cho nghiệp vụ phân tích và tổng hợp báo cáo . . . 74
16 Thiết kế class diagram cho hạ tầng phân quyền và Audit 76

Danh mục bảng biểu

1 *Danh sách thành viên và công việc* 4
2 Yêu cầu phi chức năng với ngưỡng đo lường 11
3 Driving Architecture Characteristics 12
19 Phân vùng consistency model theo domain 37

Danh sách thành viên và công việc

STT	Họ và tên	MSSV	Tổng số công việc	Hoàn thành
1	Lữ Quốc Pháp	2312562	Thiết kế class diagram, phân tích yêu cầu, thuyết trình	100%
2	Nguyễn Lê Nguyên	2312365	Hiện thực code back-end, thiết kế kiến trúc	100%
3	Lê Sĩ Nhật Quang	2312791	Phân tích yêu cầu, chọn kiến trúc, viết báo cáo	100%
4	Trần Trọng Nghĩa	2312285	Hiện thực frontend, test API backend	100%
5	Nguyễn Đức Phương	2312749	Thiết kế kiến trúc, viết báo cáo	100%

Bảng 1: ***Danh sách thành viên và công việc***

1 Bối cảnh hệ thống Intelligent Restaurant Management System

1.1 Giới thiệu và mục tiêu

Hệ thống IRMS (Intelligent Restaurant Management System) là một ứng dụng phần mềm nhằm tối ưu hóa và tự động hóa các hoạt động vận hành cốt lõi trong môi trường nhà hàng. Thông qua các giao diện số, cơ sở dữ liệu dùng chung và cơ chế điều phối dựa trên logic phần mềm, các quy trình như duyệt thực đơn, tiếp nhận và xử lý đơn hàng, quản lý trạng thái bàn, cập nhật tồn kho, cũng như tổng hợp dữ liệu vận hành được hỗ trợ một cách nhất quán. Trên cơ sở đó, sự tương tác giữa khối phục vụ (front-of-house), khối bếp (back-of-house) và bộ phận quản lý được đồng bộ hóa nhằm giảm thiểu sai lệch thông tin, hạn chế độ trễ trong luồng xử lý đơn hàng và cải thiện tính dự đoán của thời gian phục vụ.

Mục tiêu kỹ thuật của hệ thống bao gồm: (i) chuẩn hóa quy trình tiếp nhận đơn hàng và truyền đạt yêu cầu chế biến; (ii) tăng cường khả năng phối hợp giữa các trạm chế biến thông qua cơ chế định tuyến và ưu tiên công việc; (iii) nâng cao năng lực giám sát trạng thái phục vụ tại bàn và trạng thái thanh toán; và (iv) cung cấp các công cụ phân tích phục vụ ra quyết định quản trị dựa trên dữ liệu. Hệ thống được định hướng để thích ứng với các điều kiện phục vụ biến động bằng cách hỗ trợ điều chỉnh luồng công việc nhằm hạn chế tắc nghẽn và giảm thiểu lỗi tác nghiệp.

1.2 Phạm vi hệ thống và các phân hệ chức năng

Phạm vi của IRMS được giới hạn trong các chức năng vận hành chính của nhà hàng, trong đó các phân hệ sau được xem là cấu phần trọng yếu.

1.2.1 Đặt món số và quản lý thực đơn

Quá trình đặt món được hỗ trợ thông qua thiết bị đầu cuối dạng tablet hoặc terminal, nhờ đó việc ghi nhận đơn hàng được thực hiện nhanh chóng và có cấu trúc. Thực đơn có thể được cập nhật theo thời gian thực, bao gồm thay đổi giá, thiết lập

khuyến mãi và đánh dấu món tạm thời không khả dụng mà không cần khởi động lại hệ thống.

Mỗi đơn hàng được ghi nhận cùng với các thông tin tùy biến của khách, bao gồm yêu cầu đặc biệt, ghi chú dị ứng thực phẩm, lựa chọn combo và các tùy chỉnh về kích cỡ, topping hay mức độ chế biến. Hệ thống tự động phát hiện và cảnh báo khi thành phần nguyên liệu của món có khả năng xung đột với thông tin dị ứng mà khách đã cung cấp, nhằm hỗ trợ nhân viên xử lý kịp thời trước khi xác nhận đơn.

1.2.2 Hiển thị đơn bếp và điều phối luồng chế biến

Sau khi đơn hàng được xác nhận tại khu vực phục vụ, thông tin được truyền tự động đến màn hình hiển thị tại bếp mà không cần nhân viên can thiệp thủ công. Các yêu cầu chế biến được tổ chức và phân nhóm theo trạm làm việc tương ứng như nướng, chiên hay tráng miệng, giúp đội bếp nắm rõ khối lượng công việc tại từng vị trí và phân công hợp lý.

Hệ thống hỗ trợ cơ chế cảnh báo trực quan khi có đơn mới hoặc khi một yêu cầu chế biến đang tiệm cận ngưỡng thời gian mục tiêu. Đầu bếp có thể cập nhật tiến độ từng món theo các bước tuần tự từ lúc bắt đầu, đang thực hiện, cho đến khi hoàn thành và sẵn sàng phục vụ. Ngoài ra, hệ thống cho phép tạm dừng và hoàn tác trạng thái trong trường hợp cần điều chỉnh, đảm bảo tính linh hoạt trong vận hành thực tế.

1.2.3 Quản lý bàn và đặt chỗ

Trạng thái của từng bàn được theo dõi liên tục qua sơ đồ số, phản ánh đầy đủ vòng đời từ khi bàn trống, được xếp chỗ, đang gọi món, đang phục vụ, cho đến khi khách yêu cầu thanh toán và bàn cần được dọn dẹp. Khi giao dịch thanh toán hoàn tất, trạng thái bàn được cập nhật tự động mà không cần nhân viên thực hiện thêm thao tác nào.

Mô-đun đặt chỗ cho phép tiếp nhận đặt bàn theo lịch hẹn trước, theo dõi tình trạng xác nhận hoặc hủy của từng đặt chỗ, đồng thời quản lý danh sách chờ đối với khách walk-in khi nhà hàng đạt công suất. Hệ thống cũng hỗ trợ di chuyển khách giữa các bàn và gộp bàn khi cần phục vụ nhóm đông người.

1.2.4 Tính tiền, thanh toán và xuất hóa đơn

Phân hệ thanh toán tổng hợp toàn bộ các thành phần chi phí của một phiên phục vụ, bao gồm giá các món đã gọi được ghi nhận tại thời điểm đặt hàng, chiết khấu từ các chương trình khuyến mãi áp dụng, thuế VAT và tiền tip. Thứ tự tính toán được áp dụng nhất quán theo quy tắc: tổng tiền hàng trước, sau đó trừ chiết khấu, cuối cùng cộng thêm thuế, đảm bảo kết quả chính xác và có thể kiểm tra lại.

Hệ thống hỗ trợ nhiều phương thức thanh toán phổ biến bao gồm tiền mặt, thẻ tín dụng, chuyển khoản ngân hàng và ví điện tử. Đối với các thao tác nhạy cảm như hoàn tiền hay điều chỉnh hóa đơn sau khi đã xác nhận, hệ thống yêu cầu xác thực theo vai trò người dùng và ghi nhận đầy đủ vào nhật ký kiểm toán nhằm đảm bảo khả năng truy vết.

1.2.5 Giám sát tồn kho

Tồn kho nguyên liệu được cập nhật theo hai phương thức. Thứ nhất, hệ thống tự động trừ lượng nguyên liệu tương ứng theo công thức của từng món khi yêu cầu chế biến được xác nhận hoàn thành tại bếp, quá trình này diễn ra bất đồng bộ để không ảnh hưởng đến luồng phục vụ chính. Thứ hai, nhân viên có thể cập nhật thủ công lượng nguyên liệu tiêu hao hoặc hao hụt với phân loại lý do cụ thể như sơ chế, hư hỏng, điều chỉnh hay nhập thêm hàng.

Mọi thay đổi về tồn kho đều được ghi nhận vào lịch sử giao dịch để phục vụ truy vết và phân tích về sau. Hệ thống tự động phát cảnh báo khi mức tồn kho của một nguyên liệu giảm xuống dưới ngưỡng tối thiểu do quản lý cấu hình, hỗ trợ bộ phận cung ứng chủ động bổ sung trước khi xảy ra tình trạng thiếu hụt.

1.2.6 Phân tích và báo cáo

Phân hệ báo cáo cung cấp bốn loại báo cáo chính phục vụ các nhu cầu quản trị khác nhau: báo cáo doanh thu tổng hợp theo khoảng thời gian bao gồm số đơn và giá trị trung bình mỗi đơn; báo cáo thực đơn xác định top các món bán chạy nhất; báo cáo SLA bếp đánh giá tỷ lệ hoàn thành đúng hạn và thời gian chế biến trung bình theo

từng loại món; và báo cáo hiệu suất bàn phản ánh số phiên phục vụ và thời gian sử dụng trung bình của từng bàn.

Tất cả các loại báo cáo đều hỗ trợ lọc theo khoảng thời gian tùy chọn và có thể xuất ra file để phục vụ công tác kế toán, lập kế hoạch hoặc đánh giá hiệu năng định kỳ.

1.2.7 Công cụ quản trị

Hệ thống triển khai kiểm soát truy cập theo vai trò với bốn nhóm người dùng chính: quản lý có toàn quyền trên toàn bộ chức năng; nhân viên phục vụ có quyền quản lý đơn hàng, bàn và tồn kho; đầu bếp có quyền xem và cập nhật hàng đợi chế biến tại bếp; và thu ngân có quyền xử lý thanh toán. Sự phân tách quyền hạn này đảm bảo mỗi vai trò chỉ tiếp cận đúng phạm vi công việc cần thiết.

Việc cấu hình thực đơn, cập nhật giá và thiết lập chiến dịch khuyến mãi được tập trung hóa và có kiểm tra ràng buộc nghiệp vụ trước khi áp dụng — ví dụ, hệ thống không cho phép vô hiệu hóa một món đang nằm trong đơn hàng chưa hoàn thành hoặc đang là đối tượng của một chương trình khuyến mãi còn hiệu lực. Nhật ký kiểm toán được duy trì tự động cho các hành vi nhạy cảm như hoàn tiền và hủy đơn, phục vụ nhu cầu kiểm tra hậu nghiệm và tuân thủ quy trình nội bộ.

1.3 Tác nhân và tương tác vận hành điển hình

Các tác nhân chính tham gia vận hành hệ thống bao gồm bốn vai trò. Nhân viên phục vụ chịu trách nhiệm tiếp nhận khách, sắp xếp chỗ ngồi, ghi nhận đơn hàng và theo dõi trạng thái phục vụ tại từng bàn. Đầu bếp tiếp nhận và xử lý các yêu cầu chế biến tại bếp, đồng thời cập nhật tiến độ theo từng bước thực hiện. Thu ngân thực hiện tính toán hóa đơn và xử lý giao dịch thanh toán khi khách yêu cầu. Quản lý có toàn quyền giám sát vận hành, cấu hình hệ thống và xem xét các báo cáo phân tích.

Một luồng vận hành điển hình diễn ra theo trình tự sau. Nhân viên phục vụ xác nhận bàn trống và sắp xếp chỗ ngồi cho khách, từ đó mở một phiên phục vụ mới cho bàn đó. Khi khách gọi món, nhân viên ghi nhận đơn hàng trên tablet; hệ thống kiểm tra tính khả dụng của từng món cũng như lượng nguyên liệu tồn kho trước khi xác nhận đơn. Ngay sau khi đơn được xác nhận, thông tin chế biến được chuyển tự động đến

màn hình bếp mà không cần bất kỳ thao tác trung gian nào. Đầu bếp theo dõi hàng đợi tại bếp và cập nhật tiến độ từng món; khi một món hoàn thành, hệ thống tự động ghi nhận lượng nguyên liệu đã tiêu hao theo công thức tương ứng. Khi khách yêu cầu thanh toán, thu ngân tổng hợp hóa đơn và xử lý giao dịch; sau khi thanh toán hoàn tất, hệ thống tự động cập nhật trạng thái đơn hàng và đánh dấu bàn cần được dọn dẹp để chuẩn bị cho lượt phục vụ tiếp theo. Trong suốt quá trình này, dữ liệu tồn kho và các chỉ số vận hành được cập nhật liên tục, phục vụ nhu cầu giám sát và phân tích của quản lý.

1.4 Yêu cầu chức năng

Các yêu cầu chức năng cốt lõi của IRMS được tổng hợp như sau:

1. **Xem menu (FR-01):** Hiển thị thực đơn trên thiết bị của nhân viên với khả năng lọc theo trạng thái (`ACTIVE`, `OUT_OF_STOCK`) và tìm kiếm theo tên.

2. **Thực hiện order (FR-02):** Cho phép nhân viên ghi nhận đơn hàng từ khách hàng; hệ thống validate tính khả dụng của từng món và kiểm tra tồn kho nguyên liệu trước khi xác nhận.

3. **Tùy biến order (FR-03):** Cho phép nhân viên ghi chú yêu cầu đặc biệt, thông tin dị ứng và tùy chỉnh món (kích cỡ, topping, mức cay); hệ thống hiển thị cảnh báo dị ứng khi hiển thị món ăn trên menu và khi chế biến .

4. **Quản lý thực đơn (FR-04):** Cho phép CRUD món ăn, cập nhật giá và đổi trạng thái theo thời gian thực; không cho phép vô hiệu hóa món đang có trong order chưa hoàn thành hoặc đang trong chương trình khuyến mãi đang chạy.

5. **Cập nhật giá (FR-05):** Quản trị viên thay đổi giá niêm yết; giá được snapshot vào `OrderItem` tại thời điểm đặt hàng, không bị ảnh hưởng bởi thay đổi giá về sau.

6. **Khuyến mãi trên menu (FR-06):** Quản trị viên thiết lập chương trình khuyến mãi theo các hình thức: giảm theo phần trăm (`BY_PERCENT`), giảm theo số tiền cố định (`BY_AMOUNT`), combo (`COMBO`), và mua X tặng Y (`BUY_X_GET_Y`).

7. **Truyền order đến bếp (FR-07):** Hệ thống đẩy order đến KDS trong vòng 2 giây kể từ khi xác nhận, thông qua cơ chế push event (`OrderPlacedEvent`).

8. **Hiển thị hàng đợi bếp (FR-08):** Hiển thị ticket theo hàng đợi với khả năng lọc theo trạm chế biến (`station`), trạng thái (`status`), cờ sắp trễ hạn (`nearDeadline`) và sắp xếp theo deadline.

9. **Quản lý ticket bếp (FR-09):** Thông báo khi có ticket mới; cảnh báo trực quan khi ticket sắp vượt ngưỡng thời gian mục tiêu.

10. **Cập nhật trạng thái ticket (FR-10):** Đầu bếp cập nhật trạng thái ticket theo State Machine: `QUEUED` → `COOKING` → `READY` → `DELIVERED`, kèm hỗ trợ tạm dừng (`PAUSED`) và hoàn tác (`undo`).

11. **Quản lý bàn và đặt chỗ (FR-11):** Theo dõi trạng thái bàn theo vòng đời đầy đủ; hỗ trợ đặt bàn theo lịch, hàng đợi walk-in, di chuyển và gộp bàn; bàn tự động chuyển sang `DIRTY` khi thanh toán hoàn tất.

12. **Thanh toán và hóa đơn (FR-12):** Tính bill theo thứ tự: tổng tiền hàng → chiết khấu khuyến mãi → thuế VAT 10%; hỗ trợ đa phương thức thanh toán, tách hóa đơn, quản lý tip và hoàn tiền có kiểm soát quyền.

13. **Tồn kho (FR-13):** Tự động trừ kho theo công thức nguyên liệu khi ticket bếp hoàn thành; hỗ trợ cập nhật thủ công với phân loại lý do; cảnh báo tự động khi tồn kho dưới ngưỡng; ghi nhận toàn bộ giao dịch vào audit log.

14. **Phân tích và báo cáo (FR-14):** Sinh báo cáo doanh thu, món bán chạy, SLA bếp và hiệu suất bàn; lọc theo khoảng thời gian tùy chọn; hỗ trợ xuất file.

15. **Quản trị và bảo mật (FR-15):** Triển khai RBAC với bốn vai trò; duy trì audit log cho các hành vi nhạy cảm; tập trung hóa cấu hình thực đơn và khuyến mãi với cơ chế kiểm tra ràng buộc nghiệp vụ.

1.5 Yêu cầu phi chức năng

Các yêu cầu phi chức năng được xác định dựa trên benchmark ngành POS và phân tích bối cảnh hệ thống phục vụ nhà hàng quy mô vừa (∼50 bàn). Các ngưỡng dưới đây được ghi nhận là *baseline ban đầu* và sẽ được điều chỉnh sau khi có kết quả đo lường thực tế từ giai đoạn thử nghiệm.

Bảng 2: Yêu cầu phi chức năng với ngưỡng đo lường

ID	Đặc tính	Chỉ số	Ngưỡng
NFR-01	Performance	Latency order → KDS	$\leq$ 2 giây
NFR-02	Performance	Thời gian tải menu	$\leq$ 1 giây
NFR-03	Performance	Thời gian tính bill và xuất hóa đơn	$\leq$ 3 giây
NFR-04	Performance	Thời gian tải báo cáo (dataset 1.000 order)	$\leq$ 5 giây
NFR-05	Availability	Uptime trong ca làm việc	$\geq$ 99,5%
NFR-06	Reliability	mất dữ liệu tối đa	$\leq$ 1 phút
NFR-07	Reliability	thời gian khôi phục sau lỗi	$\leq$ 5 phút
NFR-08	Consistency	Độ trễ đồng bộ trạng thái order (front ↔ kitchen)	$\leq$ 2 giây
NFR-09	Consistency	Sai lệch tính toán hóa đơn	0(100% unit test)
NFR-10	Scalability	Số bàn tối đa không giảm hiệu năng	$\geq$ 50 bàn
NFR-11	Scalability	Số thiết bị đồng thời (tablet/KDS)	$\geq$ 10 thiết bị
NFR-12	Scalability	Throughput đỉnh	$\geq$ 200 order/giờ
NFR-13	Security	Thao tác nhạy cảm có audit log	100% coverage
NFR-14	Security	Token hết hạn khi không hoạt động	$\leq$ 24h
NFR-15	Security	Vi phạm RBAC trong test	0 trường hợp
NFR-16	Usability	Số bước đặt 1 món từ màn hình menu	$\leq$ 4 bước

1.6 Ràng buộc và giả định

Các ràng buộc và giả định sau được áp dụng trong phạm vi hiện tại:

- Tự động trừ kho theo công thức nguyên liệu được thực hiện bất đồng bộ khi ticket bếp hoàn thành; lỗi trong quá trình này được ghi log để xử lý thủ công

và không rollback trạng thái ticket.

- Cơ chế thông báo khách (SMS/email) được xem là khả năng tích hợp theo mức độ triển khai, chưa nằm trong phạm vi bắt buộc hiện tại.
- Luồng đặt món – điều phối bếp – thanh toán là trục nghiệp vụ chính; các năng lực phân tích nâng cao được xem là phần mở rộng.
- Hệ thống được triển khai cho một cơ sở nhà hàng duy nhất trong giai đoạn hiện tại; mở rộng đa chi nhánh là định hướng dài hạn.

1.7 Architecture Characteristics

Dựa trên phân tích yêu cầu và bối cảnh vận hành, các đặc tính kiến trúc sau được xác định là *driving characteristics* – tức là các tiêu chí quyết định hướng lựa chọn kiến trúc:

Bảng 3: Driving Architecture Characteristics

Đặc tính	**Mức ưu tiên**	**Lý do trong bối cảnh IRMS**
Performance	Cao	Độ trễ order → KDS ảnh hưởng trực tiếp đến thời gian ra món
Availability	Cao	Hệ thống ngừng hoạt động đồng nghĩa nhà hàng ngừng phục vụ
Data Consistency	Cao	Trạng thái order/bàn/thanh toán phải đồng bộ giữa các vai trò
Extensibility	Trung bình-Cao	Menu, khuyến mãi và quy tắc bếp thay đổi thường xuyên
Security	Trung bình-Cao	Thao tác tài chính cần kiểm soát quyền và audit trail

Các đặc tính sau được xem là *implicit* – phải có nhưng không phải điểm khác biệt kiến trúc: Reliability, Usability, Observability.

1.8 Tiêu chí thành công

- Độ trễ truyền đơn từ front-of-house đến KDS đạt ngưỡng NFR-01 ($\leq$ 2 giây p95) trong điều kiện giờ cao điểm.

- Tỷ lệ sai sót do thiếu hoặc nhầm thông tin tùy biến trong đơn hàng giảm về 0, nhờ cơ chế validate và snapshot thông tin tại thời điểm đặt hàng.
- Thời gian ra món được cải thiện nhờ State Machine ticket bếp và cơ chế cảnh báo `nearDeadline`.
- Báo cáo vận hành đủ chi tiết để xác định giờ cao điểm, món bán chạy và điểm tắc nghẽn tại bếp theo SLA.
- Việc mở rộng quy tắc ưu tiên bếp, thêm loại báo cáo hoặc thêm phương thức khuyến mãi được thực hiện với thay đổi tối thiểu lên các module khác.

2 Use Case Scenarios - System Modeling

Các kịch bản sử dụng (Use Case Scenarios) được sử dụng để mô tả có cấu trúc các tương tác giữa tác nhân và hệ thống trong các tình huống nghiệp vụ điển hình. Trên cơ sở đó, các luồng dữ liệu, điểm đồng bộ trạng thái, cũng như các ràng buộc vận hành (ví dụ: độ trễ, khả dụng, tính nhất quán) được làm rõ một cách rõ ràng theo quy trình. Hơn nữa, khi hệ thống được định hướng theo kiến trúc phân tán (ví dụ: Microservices), các Use Case đóng vai trò như một mô tả ranh giới domain, điểm tích hợp (integration points), và các sự kiện/trigger cần được phát sinh hoặc tiêu thụ. Do vậy, các Use Case được xem là đầu vào quan trọng để xác định module/component boundary, cơ chế giao tiếp (sync/async), và các trade-off kiến trúc liên quan.

UC1	**Xem menu (View)**
Use case	View Menu
Mục tiêu	Nhân viên xem menu theo danh mục, trạng thái món, giá hiện tại
Actors chính	Server/Front-of-house staff, Manager
Actors phụ	Hệ thống POS/Menu service
Tiền điều kiện	1. Người dùng đã đăng nhập 2. Thiết bị có kết nối mạng (hoặc có cache menu offline) 3. Menu đã được cấu hình và đang Active
Kích hoạt	Nhân viên mở màn hình Menu

Luồng chính	1. Nhân viên chọn module Menu 2. Hệ thống tải menu (danh mục, món, giá, ảnh, mô tả, option, trạng thái còn/hết) 3. Hệ thống hiển thị danh sách món theo Category (Khai vị/Món chính/Đồ uống...) 4. Nhân viên có thể: • Tìm kiếm theo tên/mã món • Lọc theo category, best-seller, đang khuyến mãi • Xem chi tiết món (mô tả, dị ứng, option) 5. Hệ thống cập nhật trạng thái "Còn/Hết" theo tồn kho hoặc cấu hình bếp
Luồng thay thế / ngoại lệ	• A1: Mất mạng → hệ thống hiển thị menu cache gần nhất + cảnh báo "Menu có thể không mới nhất" • A2: Menu trống/chưa publish → hiển thị thông báo và hướng dẫn liên hệ manager • A3: Món bị disable → món vẫn có thể hiện mờ hoặc ẩn tùy cấu hình
Hậu điều kiện	• Menu hiển thị thành công; không thay đổi dữ liệu nghiệp vụ
Quy tắc / dữ liệu	• Giá hiển thị là giá hiệu lực hiện tại (có thể đã áp dụng khuyến mãi) • Món Out of stock không thể chọn để order

UC2	**Thực hiện order (Create)**
Use case	Create Order
Mục tiêu	Server tạo order cho bàn hoặc khách mang đi và gửi vào hệ thống
Actors chính	Server/Front-of-house staff, Manager
Actors phụ	Kitchen system, Inventory (nếu trừ tồn), Billing (Có thể có tùy theo dev)
Tiền điều kiện	1. Đã đăng nhập 2. Có bàn hoặc chọn loại order (takeaway/delivery) 3. Menu có sẵn
Kích hoạt	Nhân viên bấm New Order hoặc chọn bàn → Order

Luồng chính	1. Nhân viên chọn bàn hoặc loại đơn (dine-in/takeaway) 2. Hệ thống tạo Order draft với mã tạm 3. Nhân viên chọn món từ menu, nhập số lượng 4. Hệ thống tính tạm tính (subtotal) theo giá hiện hành, hiển thị thời gian tạo 5. Nhân viên xác nhận "Gửi bếp/Place order" 6. Hệ thống: • Validate món còn bán / còn nguyên liệu (nếu có kiểm tra) • Gán số order, trạng thái New • Ghi nhận người tạo, thời điểm 7. Hệ thống đẩy order sang Kitchen system theo prep station 8. Hệ thống hiển thị order trong danh sách theo bàn
Luồng thay thế / ngoại lệ	• A1: Món hết hàng khi gửi → hệ thống báo lỗi, gợi ý món thay thế hoặc cho phép xóa món đó • A2: Order gửi thất bại do mạng → chuyển sang hàng đợi sync, hiển thị "Pending sync", tự retry • A3: 2 người order cùng một bàn → merge order
Hậu điều kiện	• Order được tạo và sẵn sàng cho bếp xử lý
Quy tắc / dữ liệu	• Order phải có ít nhất 1 món • Audit log: tạo order (ai, lúc nào, thiết bị nào)

UC3	**Note Order (Customize)**
Use case	Customize Order Item
Mục tiêu	Ghi nhận yêu cầu đặc biệt, dị ứng, combo, mức chín, thay thế nguyên liệu...
Actors chính	Server/Front-of-house staff, Manager
Actors phụ	Server
Tiền điều kiện	1. Có order draft hoặc order đang cho phép chỉnh 2. Món có option/cấu hình note
Kích hoạt	Đang trong quá trình new order hoặc chỉnh sửa ở danh sách order
Luồng chính	1. Nhân viên chọn line-item (món) cần tùy chỉnh 2. Hệ thống hiển thị: • Option có cấu trúc (size, topping, ...) • Note tự do • Allergy alert - Dị ứng 3. Nhân viên chọn option và nhập note 4. Hệ thống hiển thị preview món sau tùy chỉnh + chênh lệch giá (nếu topping tính phí) 5. Nhân viên lưu 6. Khi gửi bếp, hệ thống truyền đầy đủ option/note đến Kitchen system

Luồng thay thế / ngoại lệ	• A1: Note vượt giới hạn ký tự → hệ thống yêu cầu rút gọn • A2: Option xung đột (vd: "no ice" nhưng chọn "extra ice") → hệ thống cảnh báo và yêu cầu chọn lại • A3: Allergy flag bật → hệ thống yêu cầu xác nhận thêm ("Bạn đã xác nhận với khách?")
Hậu điều kiện	• Lưu tùy chỉnh
Quy tắc / dữ liệu	• Một số note có thể bị chặn theo policy (vd: không cho "free topping" nếu không có quyền)

UC4	**Quản lý món ăn trong menu**
Use case	Menu Item CRUD
Mục tiêu	Admin/Manager thêm, sửa, xóa (hoặc disable) món
Actors chính	Administrator/Manager
Actors phụ	Audit/Logging, Sync service tới thiết bị
Tiền điều kiện	1. Người dùng có quyền "Menu:Manage" 2. Có danh mục menu sẵn (hoặc tạo danh mục trước)
Kích hoạt	Admin mở "Menu Management"

Luồng chính (Create)	1. Admin chọn "Add item" 2. Nhập thông tin: tên, category, mô tả, ảnh, prep station, tax class, tags dị ứng, option set 3. Nhập giá mặc định 4. Chọn trạng thái: Active/Inactive 5. Lưu 6. Hệ thống validate dữ liệu và tạo món mới 7. Hệ thống publish thay đổi đến các hệ thống khác
Luồng chính (Update)	1. Admin chọn món → "Edit" 2. Sửa thuộc tính, giá 3. Lưu và publish
Luồng chính (Delete/Disable)	1. Admin chọn món → "Delete" hoặc "Disable" 2. Hệ thống kiểm tra ràng buộc: • Món có trong order đang mở? • Món có trong promo/combo đang active? 3. Nếu được phép → xóa hoặc set Inactive 4. Publish

Luồng thay thế / ngoại lệ	• A1: Không cho xóa hard-delete vì có lịch sử → hệ thống chỉ cho "Disable" • A2: Thiếu trường bắt buộc → báo lỗi cụ thể • A3: Xung đột đồng bộ (2 admin sửa) → cơ chế versioning/last-write-wins + cảnh báo
Hậu điều kiện	• Menu được cập nhật; audit log ghi nhận

UC5	**Thực hiện khuyến mãi trên menu**
Use case	Manage Promotions
Mục tiêu	Thiết lập promo theo món/combo/khung giờ/điều kiện
Actors chính	Manager/Admin
Tiền điều kiện	1. Có quyền "Promo:Manage" 2. Menu items đã tồn tại
Kích hoạt	Mở "Promotions"

Luồng chính	1. Manager chọn "Create Promotion" 2. Chọn loại: • Giảm % theo món • Giảm số tiền cố định • Combo set price • Buy X Get Y 3. Chọn danh sách món áp dụng và điều kiện (giờ, ngày, số lượng, nhóm khách...) 4. Đặt thời gian hiệu lực và ưu tiên (priority) 5. Lưu 6. Hệ thống validate xung đột promo và tính thử (simulation) 7. Publish promo đến POS
Luồng thay thế / ngoại lệ	• A1: Promo xung đột → hệ thống yêu cầu đặt priority hoặc không cho chồng • A2: Combo thiếu món thành phần → báo lỗi • A3: Promo hết hạn → tự động disable
Hậu điều kiện	• Promo hoạt động; giá hiển thị/áp vào bill theo rule

UC6	**Truyền order đến nhà bếp**
Use case	Send Order to Kitchen (Realtime)
Mục tiêu	Order mới xuất hiện trên Kitchen system gần realtime, đúng station

Actors chính	Server (người tạo), Kitchen team (người nhận)
Actors phụ	Message broker/Sync service (Nếu có)
Tiền điều kiện	1. Order đã "Placed" 2. Kitchen system online hoặc có cơ chế nhận lại khi online
Kích hoạt	Order được gửi/được update
Luồng chính	1. POS gửi event "OrderPlaced/OrderUpdated" 2. Hệ thống phân tách ticket theo prep station (grill/salad/bar...) 3. Kitchen system nhận event và hiển thị ticket mới 4. Kitchen phát âm thanh/notify theo cấu hình 5. Hệ thống ghi log thời gian "sent-to-kitchen"
Luồng thay thế / ngoại lệ	• A1: Kitchen system offline → lưu hàng đợi; khi Kitchen system online thì replay • A2: Duplicate event → Kitchen system idempotent - nhận diện "đây là event đã xử lý rồi" và bỏ qua/ghi đè. • A3: Station mapping thiếu → đưa vào "Unassigned" và cảnh báo manager
Hậu điều kiện	• Bếp nhìn thấy order và có thể bắt đầu xử lý

UC7	**Hiển thị danh sách order (Kitchen View)**
Use case	Kitchen Order Queue View
Mục tiêu	Bếp xem hàng đợi theo thời gian, theo loại món, theo station

Actors chính	Chef/Kitchen staff
Tiền điều kiện	1. Đăng nhập vào kitchen system 2. Có order đang chờ
Kích hoạt	Mở màn hình "Queue"
Luồng chính	1. Nhân viên bếp chọn chế độ xem: All / Station / Category 2. Hệ thống hiển thị danh sách ticket theo thứ tự (FIFO hoặc theo SLA - Thời gian tối đa có thể để hoàn thành 1 món) 3. Mỗi ticket hiển thị: bàn, thời gian vào bếp, danh sách món, note dị ứng, trạng thái 4. Cho phép filter: "New", "In progress", "Near deadline" 5. Auto-refresh realtime
Luồng thay thế / ngoại lệ	• A1: Quá nhiều ticket → phân trang/virtual list • A2: Lỗi sync → hiển thị trạng thái "Last updated at ..."
Hậu điều kiện	Không đổi dữ liệu (chỉ view)

UC8	**Quản lý order trong nhà bếp (Notify + Deadline)**
Use case	Kitchen Order Monitoring & Alerts
Mục tiêu	Cảnh báo order mới và order gần quá SLA/deadline
Actors chính	Kitchen lead/Chef

Tiền điều kiện	1. SLA time per item/station được cấu hình (hoặc default) 2. Kitchen system bật notifications
Kích hoạt	Order mới đến hoặc timer đạt ngưỡng cảnh báo
Luồng chính	1. Khi ticket mới đến, Kitchen system phát thông báo âm thanh + badge "New" 2. Hệ thống bắt đầu đếm timer từ "sent-to-kitchen" hoặc "start cooking" tùy rule 3. Khi gần deadline (vd: 80% SLA), ticket đổi màu và đưa lên top 4. Khi quá deadline, Kitchen system hiển thị cảnh báo mạnh hơn (màu đỏ/đẩy lên đầu) 5. Kitchen lead có thể mở ticket để xem (station nào đang chậm)
Luồng thay thế / ngoại lệ	• A1: SLA chưa cấu hình → dùng default (ví dụ 15 phút) và cảnh báo "SLA default" • A2: Ticket bị pause (do chờ khách) → cho phép "Hold" để dừng timer (nếu có quyền)
Hậu điều kiện	Trạng thái cảnh báo cập nhật

UC9	**Cập nhật trạng thái order (Kitchen Update)**
Use case	Update Cooking Status
Mục tiêu	Chef cập nhật trạng thái theo tiến trình: Bắt đầu làm → Đang nấu → Đã xong

Actors chính	Chef/Kitchen staff
Actors phụ	Server (nhận thông báo món xong), Front of House display
Tiền điều kiện	1. Ticket tồn tại và thuộc station của chef 2. Người dùng có quyền "Kitchen:UpdateStatus"
Kích hoạt	Chef thao tác trên ticket
Luồng chính	1. Chef mở ticket 2. Bấm "Start" → trạng thái ticket/item chuyển sang In progress 3. (Option) bấm "Cooking"/step tiếp theo nếu hệ thống có nhiều bước 4. Khi hoàn tất, bấm "Done" 5. Hệ thống: • ghi thời gian start/finish • cập nhật Kitchen system và gửi event về Front of House để server biết món ra 6. Front of House hiển thị "Ready to serve"
Luồng thay thế / ngoại lệ	• A1: Chef bấm nhầm "Done" → nếu policy cho phép, có "Undo" trong X giây + audit
Hậu điều kiện	• Trạng thái món và ticket cập nhật; dữ liệu phục vụ báo cáo hiệu suất

UC10	**Quản lý bàn ghế (Table Management)**

Use case	Manage Tables
Mục tiêu	Quản lý sơ đồ bàn, trạng thái bàn trống/đang phục vụ/đang chờ dọn,...
Actors chính	Host/Server/Manager
Tiền điều kiện	1. Có floor plan và danh sách bàn 2. Có quyền "Table:View" (server) hoặc "Table:Manage" (manager)
Kích hoạt	Mở "Floor plan/Tables"
Luồng chính	1. Host xem bản đồ bàn theo khu vực 2. Hệ thống hiển thị trạng thái: • Available / Seated / Ordering / Serving / Check requested / Dirty 3. Khi xếp khách: • Host chọn bàn → "Seat party" • nhập số khách, tên (tùy) • hệ thống set bàn sang Seated và bắt đầu timer 4. Server có thể mở bàn để xem order hiện có, trạng thái món, tình trạng hóa đơn 5. Manager có thể gộp/tách bàn, đổi bàn (move table), khóa bàn

Luồng thay thế / ngoại lệ	• A1: Bàn đã có khách nhưng host seat nhầm → hệ thống cảnh báo + yêu cầu quyền manager • A2: Move table khi đã có order → hệ thống chuyển toàn bộ order & bill mapping sang bàn mới
Hậu điều kiện	• Trạng thái bàn cập nhật và phản ánh trên toàn hệ thống

UC11	**Xử lý đặt bàn và hàng đợi (Reservation & Queue)**
Use case	Reservation & Waitlist
Mục tiêu	Đặt bàn theo lịch, quản lý hàng đợi, gửi thông báo cho khách
Actors chính	System
Actors phụ	Customer (nhận thông báo), Messaging service (SMS/Zalo/Email)
Tiền điều kiện	1. Có cấu hình giờ mở cửa, sức chứa, bàn 2. Tích hợp kênh nhắn tin (nếu có)
Kích hoạt	Host tạo reservation hoặc thêm waitlist
Luồng chính (Reservation)	1. Host chọn "New reservation" 2. Nhập: tên, SĐT, số khách, ngày giờ, ghi chú 3. Hệ thống gợi ý bàn phù hợp và check conflict 4. Host xác nhận 5. Hệ thống tạo reservation trạng thái "Booked" 6. (Tùy) Gửi xác nhận cho khách

Luồng chính (Waitlist)	1. Host chọn "Add to waitlist" 2. Nhập thông tin khách + party size 3. Hệ thống ước tính thời gian chờ (dựa trên turnover lịch sử + bàn trống) 4. Khi có bàn trống, host bấm "Notify" 5. Hệ thống gửi thông báo + đặt thời hạn giữ chỗ (hold time) 6. Host seat khách → chuyển waitlist entry thành seated
Luồng thay thế / ngoại lệ	• A1: Khách không đến → đánh dấu no-show; ghi log • A2: Không gửi được SMS → hiển thị lỗi và cho phép gọi tay
Hậu điều kiện	• Reservation/waitlist cập nhật; timeline phục vụ đồng bộ với bàn

UC12	**Quản lý hóa đơn (Billing & Payment)**
Use case	Bill Management & Payment
Mục tiêu	Gom món vào hóa đơn theo bàn, thanh toán đa phương thức, chia bill, tip
Actors chính	Cashier/Server
Actors phụ	Payment gateway, Bank/Wallet, Manager (refund/void)
Tiền điều kiện	1. Bàn có order đã phục vụ hoặc đang phục vụ 2. Có quyền "Bill:Checkout"

Kích hoạt	Chọn bàn → "Checkout/Payment"
Luồng chính	1. Cashier mở hóa đơn của bàn 2. Hệ thống hiển thị danh sách món, trạng thái phục vụ, subtotal, tax, discount, tip 3. Nếu khách yêu cầu chia bill: • chọn "Split" • chia theo item / theo người / theo % 4. Chọn phương thức thanh toán: thẻ / chuyển khoản / ví / tiền mặt 5. Nhập/nhận số tiền, tip 6. Hệ thống xử lý thanh toán (gọi gateway nếu online) 7. Nếu thành công: • hóa đơn trạng thái "Paid" • in/ gửi hóa đơn điện tử (tùy) • bàn chuyển trạng thái "Dirty/Cleaning" 8. Lưu audit
Luồng thay thế / ngoại lệ	• A1: Gateway fail → cho retry hoặc đổi phương thức • A2: Thanh toán một phần (partial) → hóa đơn "Partially paid", còn balance • A3: Refund/Void → yêu cầu quyền manager + lý do + audit log
Hậu điều kiện	• Doanh thu ghi nhận, dữ liệu báo cáo cập nhật

UC13	**Quản lý số lượng hàng hóa (Inventory)**
Use case	Ingredient Usage & Low-stock Alerts
Mục tiêu	Nhân viên nhập lượng sử dụng nguyên liệu, cảnh báo thấp, dự đoán reorder
Actors chính	Kitchen staff/Manager
Actors phụ	Reporting/Forecasting service
Tiền điều kiện	1. Có danh mục nguyên liệu + mức min threshold 2. Có mapping món → nguyên liệu (nếu muốn tự động trừ)
Kích hoạt	Nhập usage hoặc hệ thống tự trừ theo order
Luồng chính (Manual usage)	1. Nhân viên mở module Inventory → "Record usage" 2. Chọn nguyên liệu, nhập số lượng dùng và đơn vị 3. Chọn lý do (prep/spoilage/adjustment) 4. Lưu 5. Hệ thống cập nhật tồn kho, ghi audit
Luồng chính (Low-stock alert)	1. Khi tồn < threshold, hệ thống gửi cảnh báo tới manager 2. Manager xem danh sách cần reorder
Luồng chính (Forecast reorder)	1. Hệ thống tổng hợp lịch sử usage/sales 2. Đưa ra dự báo lượng cần đặt cho kỳ tới (ngày/tuần) 3. Manager xuất danh sách reorder

Luồng thay thế / ngoại lệ	• A1: Nhập sai đơn vị → hệ thống yêu cầu quy đổi hoặc từ chối • A2: Không đủ quyền điều chỉnh tồn → chỉ manager được chỉnh
Hậu điều kiện	• Tồn kho cập nhật; cảnh báo & dự báo sẵn sàng

UC14	**Phân tích và báo cáo (Analytics & Reporting)**
Use case	Sales & Operations Reporting
Mục tiêu	Báo cáo doanh thu, giờ cao điểm, best-seller, bottleneck, hiệu suất nhân sự; xuất báo cáo
Actors chính	Administrator/Manager
Tiền điều kiện	1. Có quyền "Reports:View" 2. Có dữ liệu giao dịch
Kích hoạt	Mở module "Reports"

Luồng chính	1. Manager chọn loại báo cáo: Sales, Menu performance, Labor, Kitchen SLA, Table turnover 2. Chọn khoảng thời gian, chi nhánh, ca làm 3. Hệ thống hiển thị dashboard: • doanh thu theo giờ/ngày • top món bán chạy • ticket time trung bình theo station • tỷ lệ hủy/void/refund 4. Manager chọn "Export" (CSV/PDF) 5. Hệ thống tạo file và cho tải về (hoặc gửi email)
Luồng thay thế / ngoại lệ	• A1: Khoảng thời gian quá lớn → hệ thống chạy nền và thông báo khi xong • A2: Không có dữ liệu → hiển thị trạng thái empty
Hậu điều kiện	• Báo cáo được xem/xuất; không ảnh hưởng vận hành

UC15	**Kiểm soát an toàn và quyền truy cập (RBAC - Kiểm soát truy cập theo vai trò + Audit logs)**
Use case	Access Control & Audit Logs
Mục tiêu	Phân quyền theo role; audit các hành động nhạy cảm (hủy đơn, hoàn tiền...)
Actors chính	Administrator
Actors phụ	Manager (xử lý nghiệp vụ nhạy cảm), Auditor

Tiền điều kiện	1. Hệ thống có danh sách role: managers, servers, chefs, cashiers 2. Người dùng được gán role
Kích hoạt	User thao tác chức năng cần quyền hoặc admin xem audit
Luồng chính (RBAC) - Role-based access control:	1. User thực hiện hành động (vd: refund) 2. Hệ thống kiểm tra quyền theo role + policy 3. Nếu đủ quyền → cho phép thao tác 4. Nếu không đủ → chặn và hiển thị thông báo "Access denied"
Luồng chính (Audit log)	1. Khi hành động nhạy cảm xảy ra (void/refund/discount override/price change): 2. Hệ thống ghi audit: userId, role, action, before/after, timestamp, reason, device 3. Admin mở "Audit logs", filter theo thời gian/user/action 4. Admin export log nếu cần
Luồng thay thế / ngoại lệ	• A1: Manager override: server không có quyền → yêu cầu manager nhập PIN/approve • A2: Audit storage lỗi → vẫn cho phép/hoặc chặn tùy mức độ nghiêm trọng
Hậu điều kiện	• Quyền truy cập được đảm bảo; truy vết đầy đủ

3 So sánh và lựa chọn phong cách kiến trúc phù hợp

Việc chọn architecture style cho IRMS được tiếp cận như một bài toán trade-off giữa các *architecture characteristics* đã xác định ở mục trên. IRMS vừa có nhiều domain chức năng (menu, order, kitchen workflow, table/reservation, billing, inventory, reporting) vừa yêu cầu vận hành trơn chu giữa front-of-house và kitchen theo thời gian gần thực. Do vậy, các tiêu chí performance, availability, data consistency, extensibility và security được ưu tiên khi so sánh các style.

3.1 Ước tính tải hệ thống

Trước khi so sánh các style, nhóm thực hiện ước tính tải để xác định quy mô bài toán. Với nhà hàng ∼50 bàn, trung bình 4 khách/bàn, thời gian phục vụ ∼90 phút/lượt, trong giờ cao điểm khoảng 30 bàn hoạt động đồng thời. Mỗi bàn phát sinh 5–8 nghiệp vụ request (đặt món, cập nhật trạng thái ticket, thanh toán), tương đương **150–240 request nghiệp vụ/giờ**. Khi tính thêm các request kỹ thuật (heartbeat, sync, poll), tổng lên khoảng **800–1.200 request/giờ** trong điều kiện đỉnh tải. Đây là mức tải *vừa phải*, hoàn toàn trong khả năng xử lý của một Modular Monolith được tối ưu tốt, và là cơ sở để đánh giá các style dưới đây.

3.2 Modular Monolith

Modular Monolith có điểm mạnh ở simplicity và chi phí vận hành thấp: build nhanh, debug dễ, deploy một lần thay vì quản lý nhiều service. Tuy nhiên, kiến trúc này bị hạn chế ở scalability và fault-tolerance. Với IRMS, nếu toàn bộ hệ thống chạy chung một khối (không phân tách rõ domain boundary), khi một module (ví dụ kitchen queue hoặc payment) gặp sự cố hoặc quá tải, hiệu ứng lan truyền có thể làm giảm availability của toàn hệ thống. Đây là trade-off quan trọng cần giải quyết ở cấp thiết kế nội bộ.

3.3 Microservices

Microservices được đánh giá cao về maintainability, deployability, scalability, fault-tolerance và evolvability. Điều này phù hợp về mặt lý thuyết với IRMS vì các domain có vòng đời thay đổi khác nhau: menu/promotion thường xuyên update, kitchen workflow cần tối ưu liên tục, billing/payment có tính nhạy cảm cao. Khi tách thành các service độc lập, việc scale từng phần chủ động hơn (scale riêng kitchen hoặc ordering trong giờ cao điểm). Tuy nhiên, với mức tải ước tính 800–1.200 request/giờ, overhead vận hành của microservices (nhiều service, nhiều API call giữa service, cần infrastructure phức tạp cho service discovery và distributed tracing) là không tương xứng với lợi ích thu được ở giai đoạn hiện tại.

3.4 Event-Driven

Event-driven architecture nổi bật ở responsiveness, fault-tolerance và evolvability. Với IRMS, nhiều tình huống tự nhiên đã mang tính sự kiện: `OrderPlacedEvent`, `KitchenItemDoneEvent`, `PaymentCompletedEvent`, `OrderPaidEvent`, cảnh báo tồn kho thấp, audit log... Nếu các tương tác liên domain này được xử lý bằng event (publish/subscribe) thay vì direct call đồng bộ, các component giảm coupling, giảm rủi ro blocking khi một service chậm, và dễ mở rộng thêm hành vi mà không cần sửa code các module hiện có.

Trade-off là hệ thống phức tạp hơn ở tính nhất quán và xử lý lỗi. Cụ thể, không phải mọi domain đều có thể chấp nhận eventual consistency:

3.5 Kết luận lựa chọn: Modular Monolith kết hợp Event-Driven có chọn lọc

Với bối cảnh nhà hàng quy mô vừa (∼50 bàn, tải đỉnh ∼800–1.200 request/giờ), kiến trúc được lựa chọn là **Modular Monolith kết hợp Event-Driven có chọn lọc**.

Bảng 19: Phân vùng consistency model theo domain

Domain	Consistency Model	Lý do
Trạng thái thanh toán	Strong consistency	Sai sót tài chính không thể chấp nhận
Trạng thái order (front ↔ kitchen)	Strong consistency	Lệch thông tin gây sai món
Trạng thái bàn	Eventual consistency	Độ trễ 1–2 giây chấp nhận được
Tự động trừ kho	Eventual consistency	Xử lý bất đồng bộ, log để xử lý thủ công nếu lỗi
Analytics & báo cáo	Eventual consistency	Xử lý batch, không cần realtime

3.5.1 Ranh giới Modular Monolith

Hệ thống được tổ chức thành một Modular Monolith với tám domain được phân tách rõ theo boundary: `order`, `menu`, `kitchen`, `inventory`, `payment`, `promotion`, `seating`, `report`. Mỗi domain tuân theo cấu trúc lớp nhất quán (controller / service interface / service impl / repository / entity / dto / event / listener) và không được phép gọi trực tiếp vào implementation của domain khác. Giao tiếp nội domain đi qua service interface; giao tiếp cross-domain đi qua Facade Layer hoặc Event Bus.

Facade Layer (`orchestration/`) đóng vai trò điều phối các use case yêu cầu nhiều domain: `OrderingFacade` (validate món + kho → tạo order), `AdminCatalogFacade` (kiểm tra ràng buộc → thay đổi menu/promo), `CheckoutFacade` (tính bill với discount + thuế), `ReportingDataFacade` (tổng hợp dữ liệu từ nhiều domain cho báo cáo).

3.5.2 Phạm vi áp dụng Event-Driven

Event-Driven được áp dụng có chọn lọc cho các workflow liên domain và realtime, nơi mà việc gọi đồng bộ trực tiếp sẽ tạo ra coupling không cần thiết:

- `OrderPlacedEvent`: Order domain → Kitchen domain (tạo ticket cho từng `OrderItem`)
- `KitchenItemDoneEvent`: Kitchen domain → Inventory domain (tự động trừ kho theo công thức, chạy `@Async`)

- `PaymentCompletedEvent`: Payment domain → Order domain (`markAsPaid()`) và Seating domain (`clearTable()`)
- `OrderPaidEvent`: Order domain → Seating domain (bàn chuyển sang `DIRTY`)

Các luồng yêu cầu strong consistency (tính tiền, validate kho trước khi đặt món) vẫn sử dụng direct synchronous call qua Facade để đảm bảo tính toàn vẹn dữ liệu.

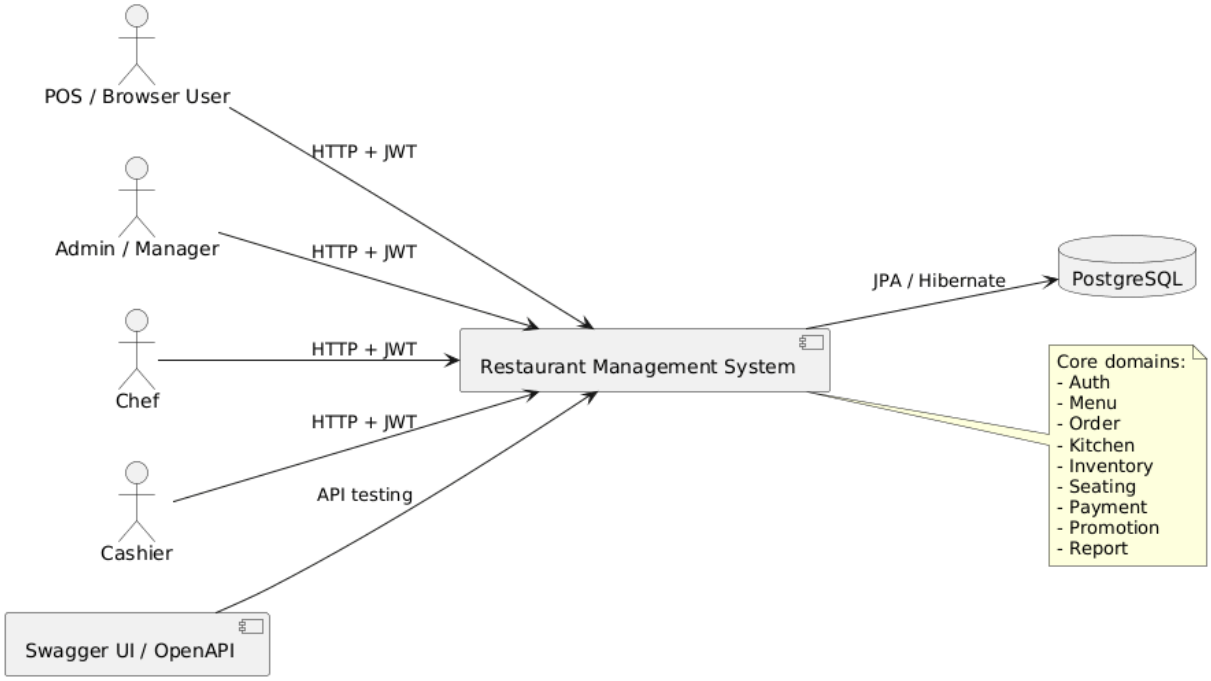
3.5.3 Lý do không chọn Microservices ngay từ đầu

Việc triển khai Microservices ở giai đoạn hiện tại sẽ tạo ra độ phức tạp vận hành không tương xứng: cần infrastructure cho service discovery, distributed tracing, saga pattern cho distributed transaction, và tăng đáng kể số lượng network hop. Với mức tải hiện tại (∼200 order/giờ nghiệp vụ), lợi ích của Microservices chưa đủ bù đắp overhead này.

4 Xây dựng kiến trúc phần mềm cho hệ thống

4.1 Module Views

4.1.1 Context View



Hình 1: Context View của hệ thống

Sơ đồ Context View mô tả mối quan hệ tổng quát giữa hệ thống quản lý nhà hàng và các tác nhân bên ngoài. Hệ thống đóng vai trò trung tâm, tiếp nhận yêu cầu từ nhiều nhóm người dùng khác nhau thông qua giao thức HTTP kết hợp cơ chế xác thực JWT.

POS / Browser User: Người dùng thông thường sử dụng hệ thống thông qua giao diện POS hoặc trình duyệt web để thực hiện các chức năng như xem thực đơn, đặt món và thanh toán.

Admin / Manager: Quản trị viên hoặc quản lý sử dụng hệ thống để quản lý menu, khuyến mãi, báo cáo doanh thu và giám sát hoạt động nhà hàng.

Chef: Đầu bếp nhận thông tin đơn hàng từ hệ thống để xử lý và cập nhật trạng thái món ăn.

Cashier: Thu ngân sử dụng hệ thống để xử lý thanh toán và xác nhận giao dịch.

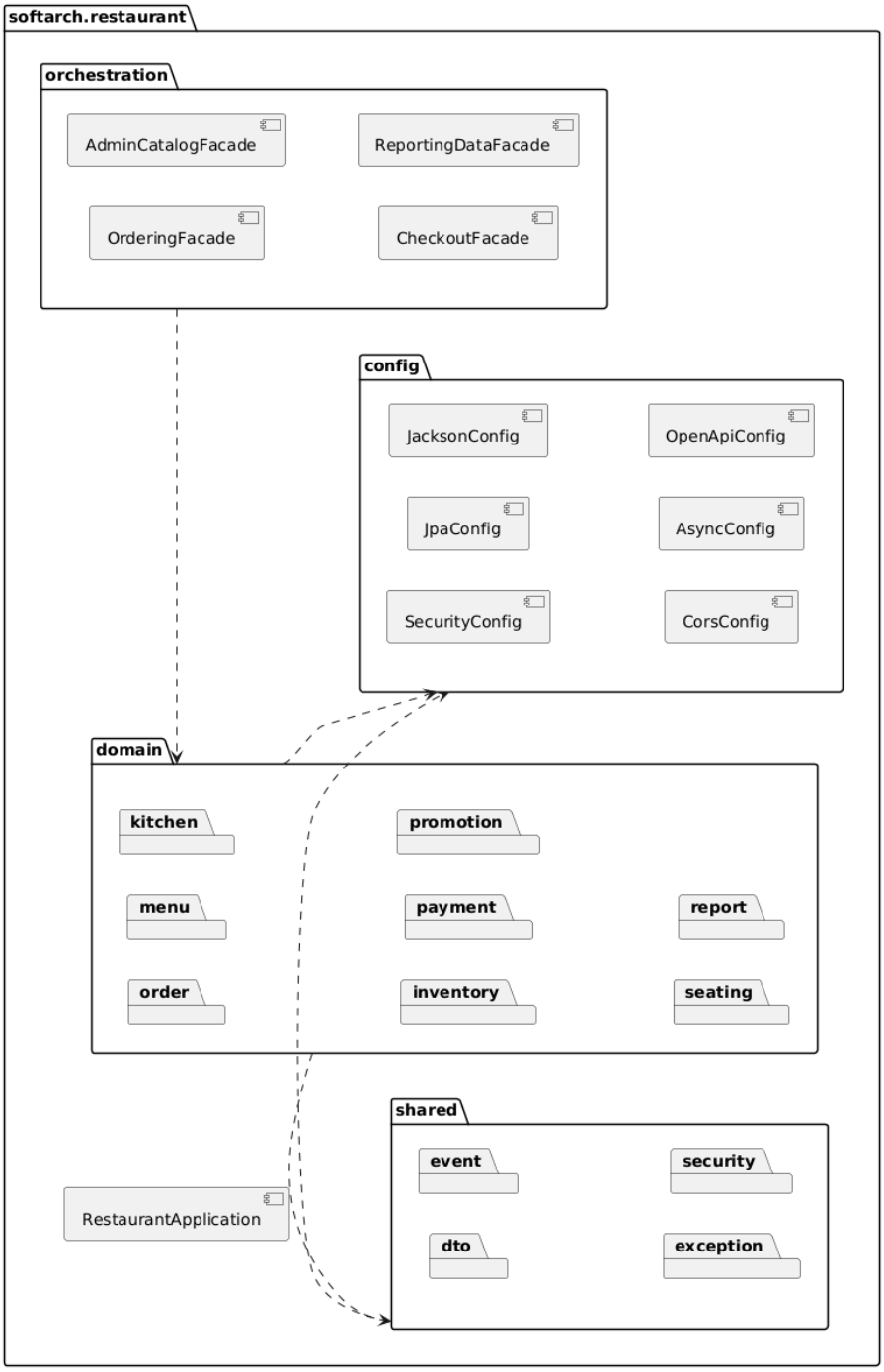
Restaurant Management System: Là hệ thống trung tâm chứa các domain nghiệp vụ chính bao gồm: *Auth, Menu, Order, Kitchen, Inventory, Seating, Payment, Promotion, Report*.

PostgreSQL: Được sử dụng làm hệ quản trị cơ sở dữ liệu nhằm lưu trữ dữ liệu nghiệp vụ của hệ thống thông qua cơ chế JPA/Hibernate.

Swagger UI / OpenAPI: Hỗ trợ kiểm thử và tài liệu hóa RESTful API trong quá trình phát triển và tích hợp hệ thống.

Luồng giao tiếp: Người dùng gửi request → Restaurant Management System → PostgreSQL.

4.1.2 Package View



Hình 2: Package View của hệ thống

Sơ đồ Package View mô tả cấu trúc tổ chức mã nguồn của hệ thống theo các package chức năng nhằm tăng tính phân tách trách nhiệm, khả năng mở rộng và bảo trì.

Package config: Chứa các thành phần cấu hình hệ thống như: *SecurityConfig,*

JpaConfig, JacksonConfig, CorsConfig, AsyncConfig, OpenApiConfig. Các package này hỗ trợ cấu hình bảo mật, ORM, serialization, CORS, xử lý bất đồng bộ và tài liệu API.

Package shared: Chứa các thành phần dùng chung trong toàn hệ thống bao gồm: *dto, event, exception, security.* Các thành phần này giúp tái sử dụng mã nguồn và giảm trùng lặp logic.

Package orchestration: Chứa các facade chịu trách nhiệm điều phối luồng nghiệp vụ giữa nhiều domain khác nhau như: *OrderingFacade, CheckoutFacade, AdminCatalogFacade, ReportingDataFacade.*

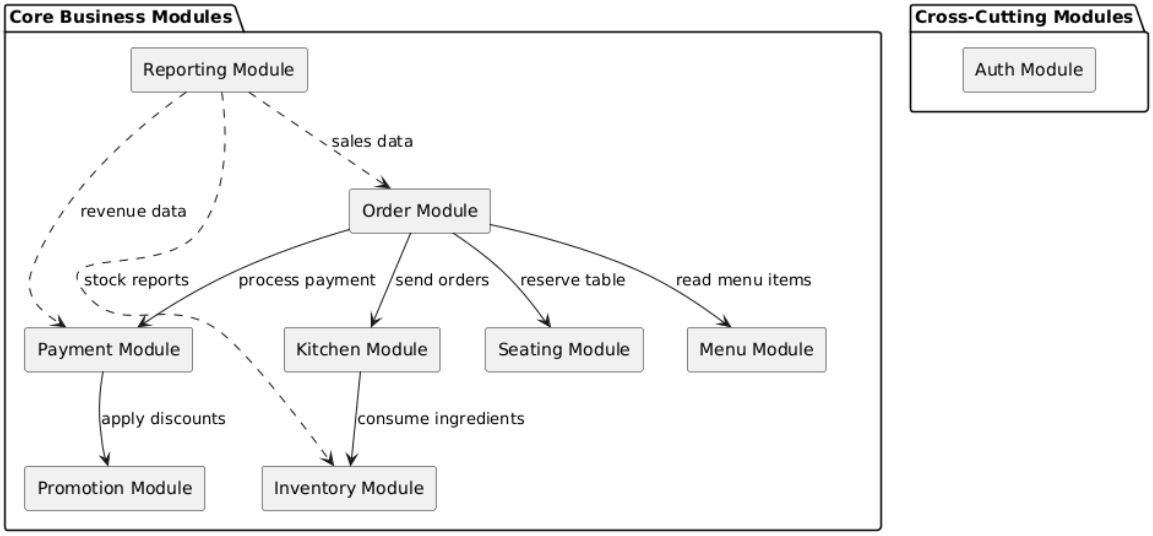
Package domain: Chứa các domain nghiệp vụ chính của hệ thống: *order, menu, kitchen, inventory, payment, promotion, seating, report.* Mỗi domain được tổ chức độc lập nhằm đảm bảo tính module hóa và giảm coupling giữa các thành phần.

RestaurantApplication: Là điểm khởi chạy chính của toàn bộ hệ thống Spring Boot.

Quan hệ phụ thuộc:

- Package *shared* phụ thuộc vào *config* để sử dụng các cấu hình hệ thống.
- Package *orchestration* phụ thuộc vào *domain* để điều phối nghiệp vụ.
- Package *domain* sử dụng các thành phần dùng chung từ *shared.*
- Package *domain* đồng thời phụ thuộc vào *config* cho các cấu hình hạ tầng.

4.1.3 Module View



Hình 3: Module View của hệ thống

Sơ đồ Module View mô tả cấu trúc phân rã hệ thống theo các business module chính nhằm tăng tính độc lập, khả năng mở rộng và tái sử dụng.

Hệ thống được chia thành hai nhóm module chính:

Core Business Modules: Bao gồm các module nghiệp vụ cốt lõi của hệ thống:

- *Menu Module*: quản lý thực đơn và thông tin món ăn.
- *Order Module*: xử lý đặt món và quản lý đơn hàng.
- *Kitchen Module*: quản lý hàng chờ chế biến món ăn.
- *Payment Module*: xử lý thanh toán và giao dịch.
- *Inventory Module*: quản lý nguyên liệu và tồn kho.
- *Promotion Module*: quản lý khuyến mãi và giảm giá.
- *Seating Module*: quản lý bàn và đặt chỗ.
- *Reporting Module*: tổng hợp báo cáo và thống kê doanh thu.

Cross-Cutting Modules: Bao gồm các module hỗ trợ xuyên suốt toàn hệ thống:

- *Auth Module*: cung cấp cơ chế xác thực và phân quyền người dùng thông qua JWT Security.

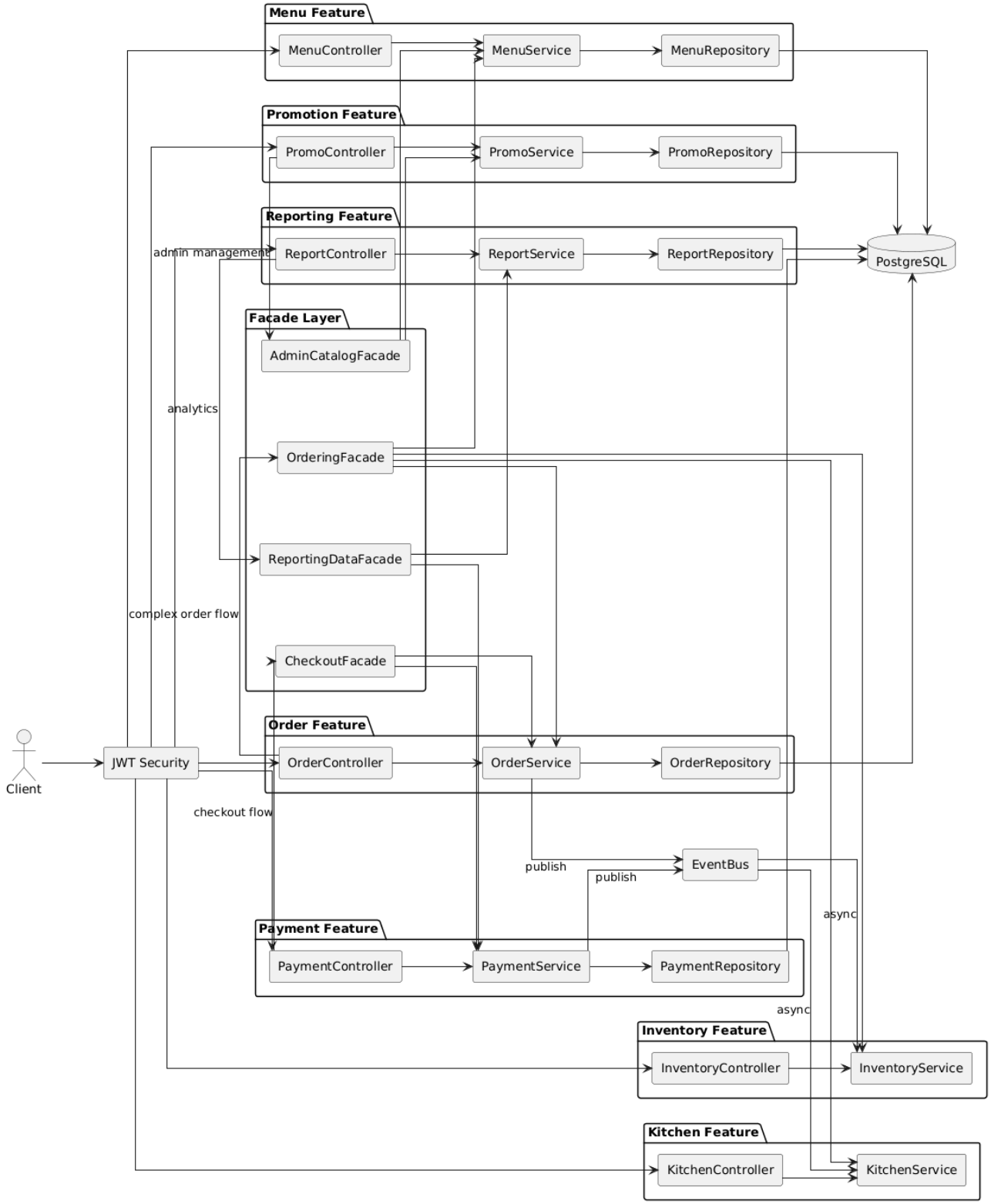
Quan hệ giữa các module:

- *Order Module* sử dụng dữ liệu từ *Menu Module* để tạo đơn hàng.
- *Order Module* tương tác với *Payment Module* để xử lý thanh toán.
- *Order Module* gửi thông tin đơn hàng tới *Kitchen Module*.
- *Kitchen Module* cập nhật và tiêu thụ nguyên liệu từ *Inventory Module*.
- *Payment Module* áp dụng khuyến mãi từ *Promotion Module*.
- *Order Module* tương tác với *Seating Module* để quản lý đặt bàn.
- *Reporting Module* thu thập dữ liệu từ các module nghiệp vụ nhằm tạo báo cáo doanh thu, thống kê đơn hàng và tình trạng tồn kho.

Sơ đồ Module View giúp mô tả rõ ràng cấu trúc nghiệp vụ của hệ thống và mối quan hệ giữa các business module chính trong kiến trúc phần mềm.

4.2 Component & Connector Views

4.2.1 Core Business Flow Component View



Hình 4: Core Business Flow Component View

Mô tả

Sơ đồ Core Business Flow Component View mô tả luồng xử lý nghiệp vụ chính của hệ thống từ tầng controller cho tới service, repository và facade orchestration.

Mục tiêu của sơ đồ này là thể hiện:

- Các feature chính của hệ thống.
- Quan hệ giữa controller, service và repository.
- Vai trò của facade trong các workflow phức tạp.
- Luồng xử lý chính giữa các business component.

Hệ thống được tổ chức theo từng feature riêng biệt như: *Order Feature, Menu Feature, Payment Feature, Kitchen Feature, Inventory Feature, Promotion Feature* và *Reporting Feature*.

Mỗi feature bao gồm:

- Controller tiếp nhận HTTP request.
- Service xử lý business logic.
- Repository thao tác với dữ liệu.

Ngoài ra, hệ thống còn sử dụng:

- **JWT Security** để xác thực request.
- **Facade Layer** để orchestration giữa nhiều service khi xử lý workflow phức tạp.
- **EventBus** để hỗ trợ giao tiếp bất đồng bộ.

Luồng hoạt động

Quá trình xử lý nghiệp vụ chính diễn ra theo các bước:

1. Client gửi request tới hệ thống.
2. JWT Security thực hiện xác thực và phân quyền.
3. Controller tiếp nhận request.

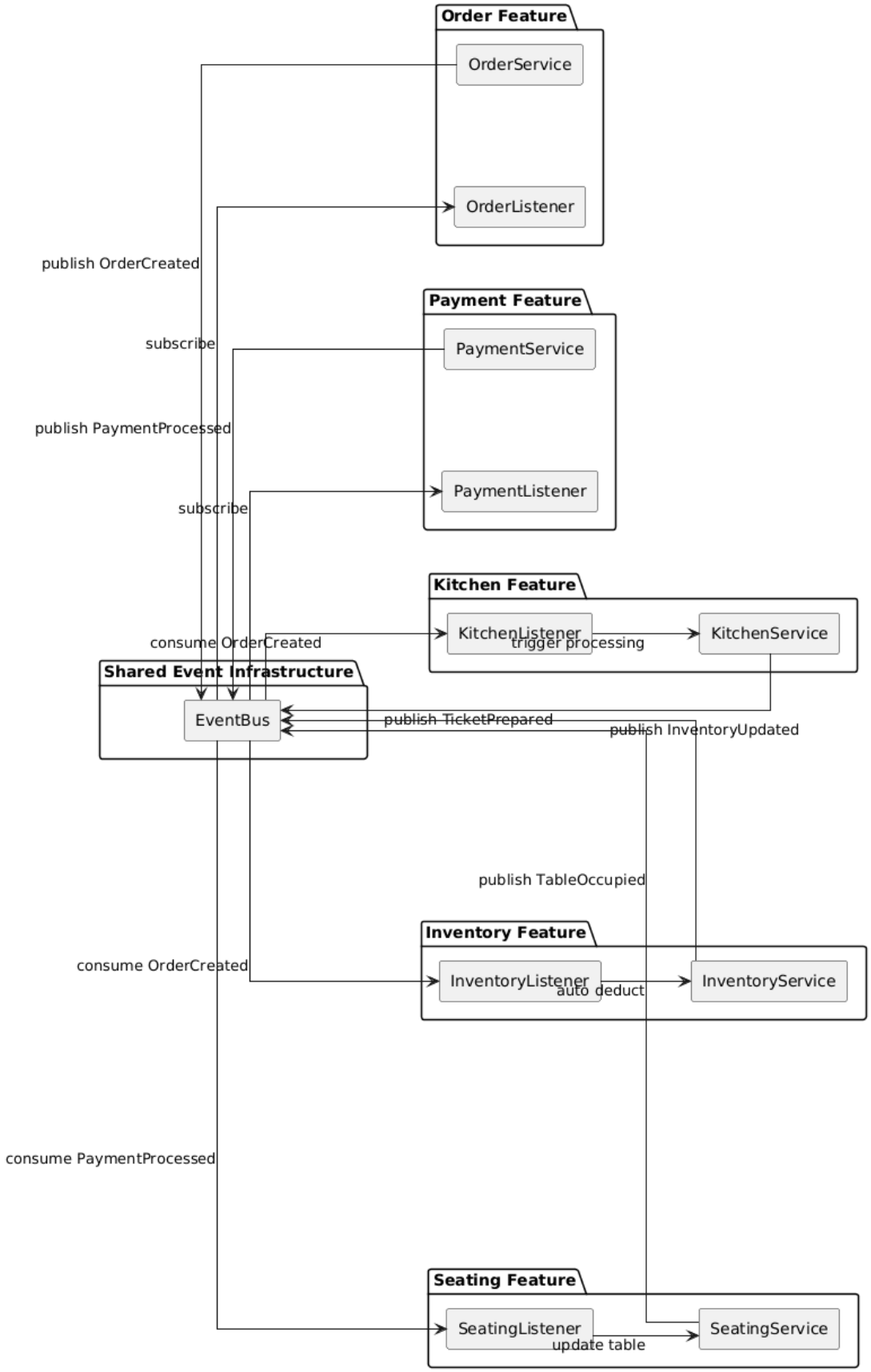
4. Với các tác vụ đơn giản, controller gọi trực tiếp service tương ứng.
5. Với các workflow phức tạp, controller sẽ gọi facade để orchestration giữa nhiều service.
6. Service xử lý business logic và thao tác với repository.
7. Repository truy cập dữ liệu trong PostgreSQL.
8. Một số service có thể publish event tới EventBus để kích hoạt các tác vụ bất đồng bộ.

Guideline

Khi xây dựng Core Business Flow Component View cần tuân thủ các nguyên tắc sau:

- Tập trung mô tả luồng nghiệp vụ chính của hệ thống.
- Mỗi feature nên được tách biệt rõ ràng nhằm tăng modularity.
- Controller cần giữ vai trò "thin controller".
- Business logic phải nằm trong service layer.
- Repository chỉ thực hiện persistence.
- Các workflow phức tạp nên sử dụng facade để orchestration.
- Không đưa implementation detail ở mức method hoặc entity field vào sơ đồ.
- Luồng xử lý chính cần dễ đọc và dễ theo dõi.

4.2.2 Event-Driven Component View



Hình 5: Event-Driven Component View

Mô tả

Sơ đồ Event-Driven Component View mô tả cơ chế giao tiếp bất đồng bộ giữa các feature thông qua EventBus và các event listener.

Kiến trúc event-driven giúp:

- Giảm coupling giữa các module.
- Tăng khả năng mở rộng hệ thống.
- Hỗ trợ asynchronous processing.
- Cho phép nhiều feature phản ứng với cùng một domain event.

Các service như: *OrderService, PaymentService, KitchenService, InventoryService* và *SeatingService* có khả năng publish domain events tới EventBus.

Các listener sẽ subscribe các event tương ứng để xử lý follow-up actions.

Luồng hoạt động

Ví dụ luồng xử lý event-driven:

1. OrderService publish sự kiện *OrderCreated*.
2. EventBus dispatch event tới các listener liên quan.
3. KitchenListener nhận event và gửi ticket xuống bếp.
4. InventoryListener nhận event và tự động trừ nguyên liệu.
5. Sau khi thanh toán thành công, PaymentService publish event *PaymentProcessed*.
6. SeatingListener nhận event và cập nhật trạng thái bàn.

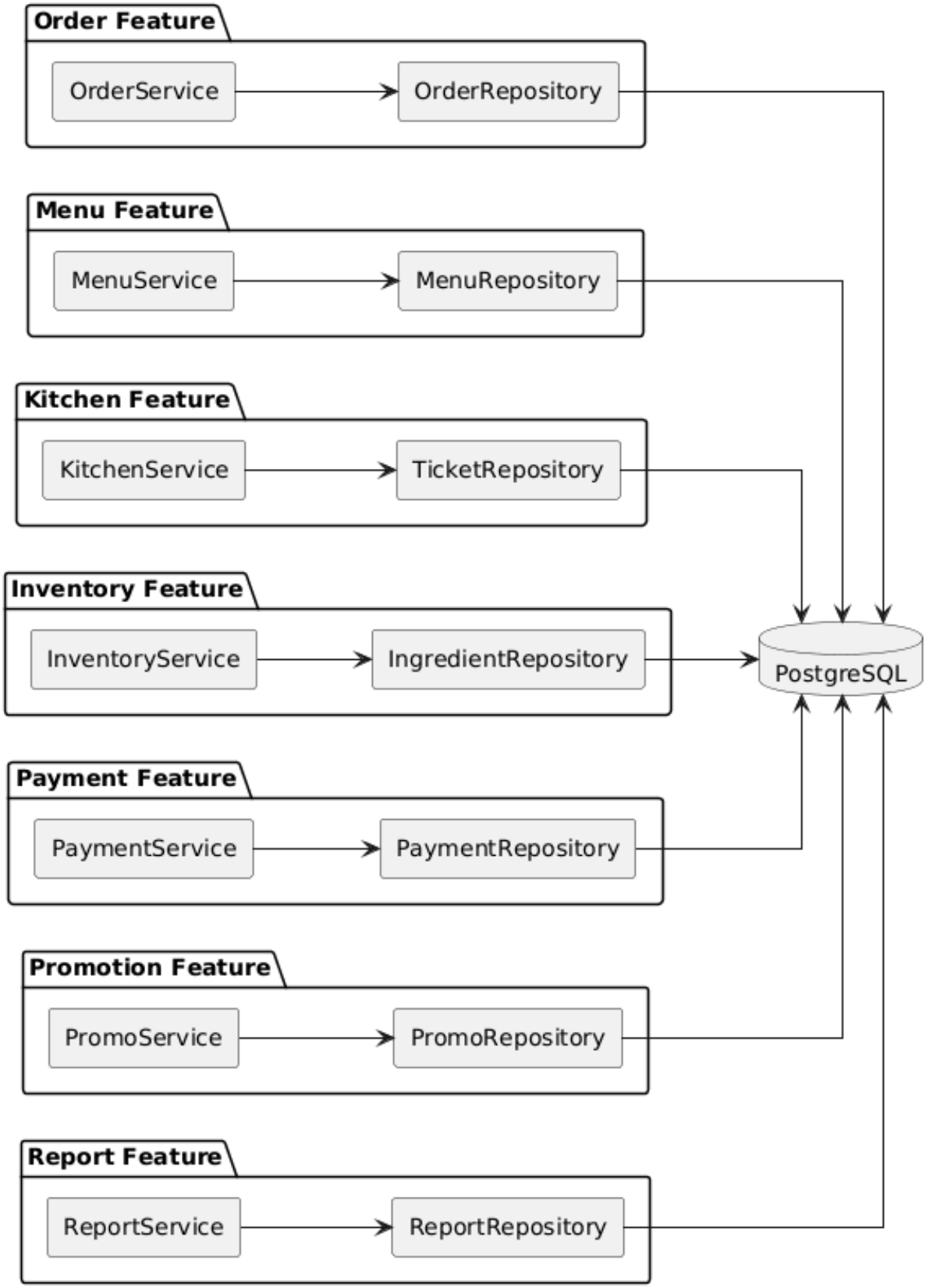
Guideline

Khi xây dựng Event-Driven Component View cần tuân thủ các nguyên tắc sau:

- Chỉ mô tả các interaction bất đồng bộ.
- Event cần được đặt tên rõ ràng theo domain event.

- Listener nên xử lý tác vụ độc lập và không phụ thuộc trực tiếp vào publisher.
- Không tạo dependency trực tiếp giữa các feature thông qua method call nếu có thể dùng event.
- EventBus cần đóng vai trò trung gian giao tiếp.
- Tránh đưa logic xử lý chi tiết của listener vào sơ đồ.
- Các asynchronous flow cần dễ dàng nhận biết.

4.2.3 Persistence Component View



Hình 6: Persistence Component View

Mô tả

Sơ đồ Persistence Component View mô tả cách các service tương tác với repository và cơ sở dữ liệu PostgreSQL.

Mục tiêu của sơ đồ này là:

- Thể hiện kiến trúc persistence layer.
- Mô tả dependency giữa service và repository.
- Minh họa cách dữ liệu được lưu trữ trong hệ thống.

Mỗi feature đều có repository riêng nhằm đảm bảo:

- Tính độc lập giữa các domain.
- Separation of Concerns.
- Dễ mở rộng và bảo trì.

Tất cả repository đều kết nối tới PostgreSQL thông qua JPA/Hibernate.

Luồng hoạt động

Quá trình persistence diễn ra như sau:

1. Service xử lý business logic.
2. Service gọi repository tương ứng để thao tác dữ liệu.
3. Repository thực hiện CRUD operation thông qua ORM framework.
4. Dữ liệu được lưu trữ hoặc truy xuất từ PostgreSQL.

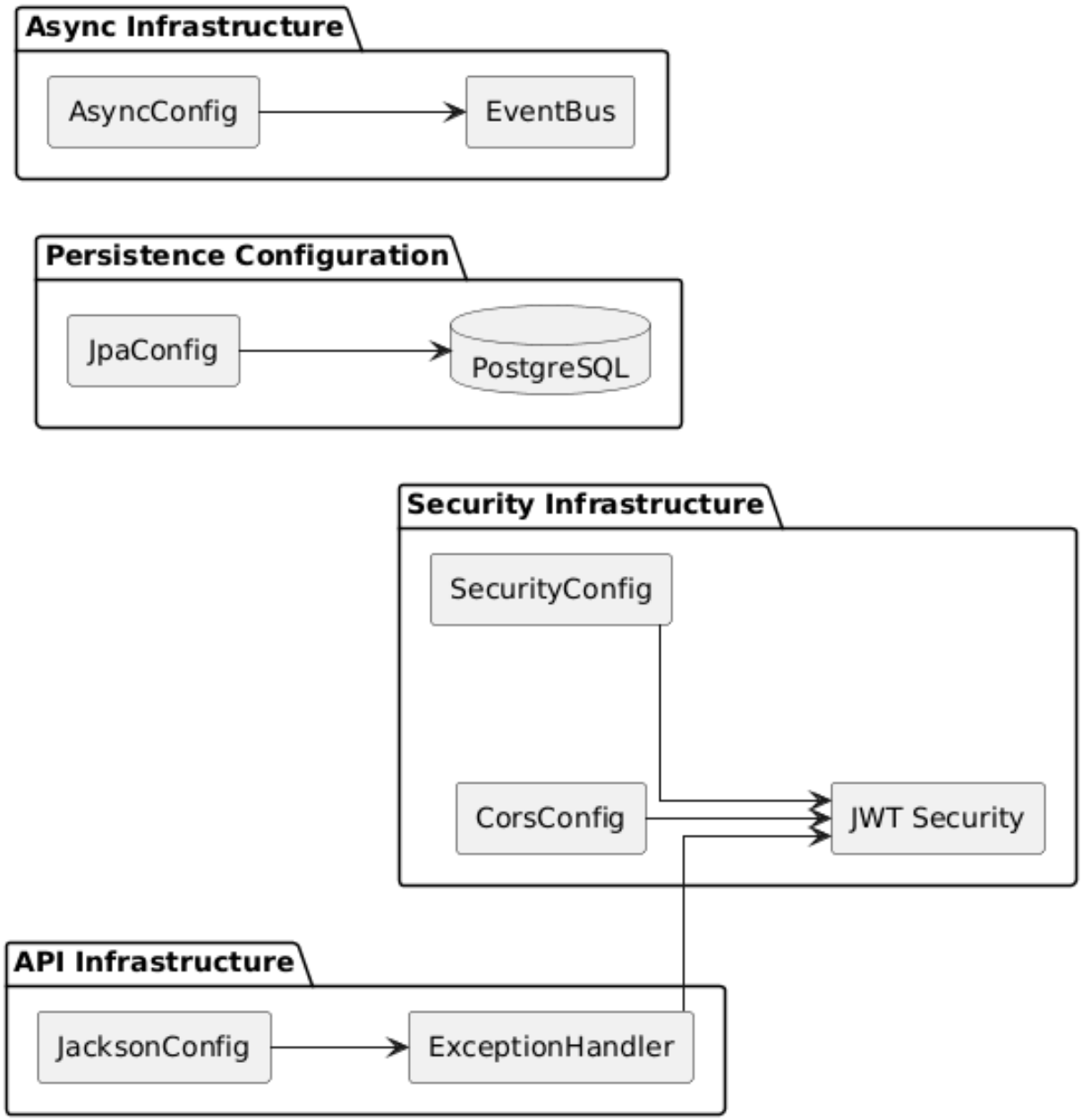
Guideline

Khi xây dựng Persistence Component View cần tuân thủ các nguyên tắc sau:

- Repository chỉ chịu trách nhiệm persistence.
- Không đặt business logic trong repository.
- Mỗi domain nên có repository riêng.
- Các dependency giữa service và database phải đi qua repository layer.

- Không để controller truy cập trực tiếp database.
- Persistence layer cần được tách biệt khỏi presentation layer.
- Chỉ mô tả dependency persistence ở mức component.

4.2.4 Infrastructure / Technical Component View



Hình 7: Infrastructure / Technical Component View

Mô tả

Sơ đồ Infrastructure / Technical Component View mô tả các thành phần kỹ thuật và cross-cutting concerns hỗ trợ cho toàn bộ hệ thống.

Các thành phần này không chứa business logic nhưng đóng vai trò quan trọng trong việc vận hành hệ thống.

Sơ đồ bao gồm các nhóm thành phần chính:

- **Security Infrastructure**
  - JWT Security
  - SecurityConfig
  - CorsConfig
- **Persistence Configuration**
  - JpaConfig
  - PostgreSQL
- **Async Infrastructure**
  - AsyncConfig
  - EventBus
- **API Infrastructure**
  - ExceptionHandler
  - JacksonConfig

Luồng hoạt động

Các technical component hỗ trợ hệ thống như sau:

1. SecurityConfig cấu hình JWT authentication và authorization.
2. CorsConfig quản lý CORS policy cho API.
3. JpaConfig cấu hình kết nối cơ sở dữ liệu.

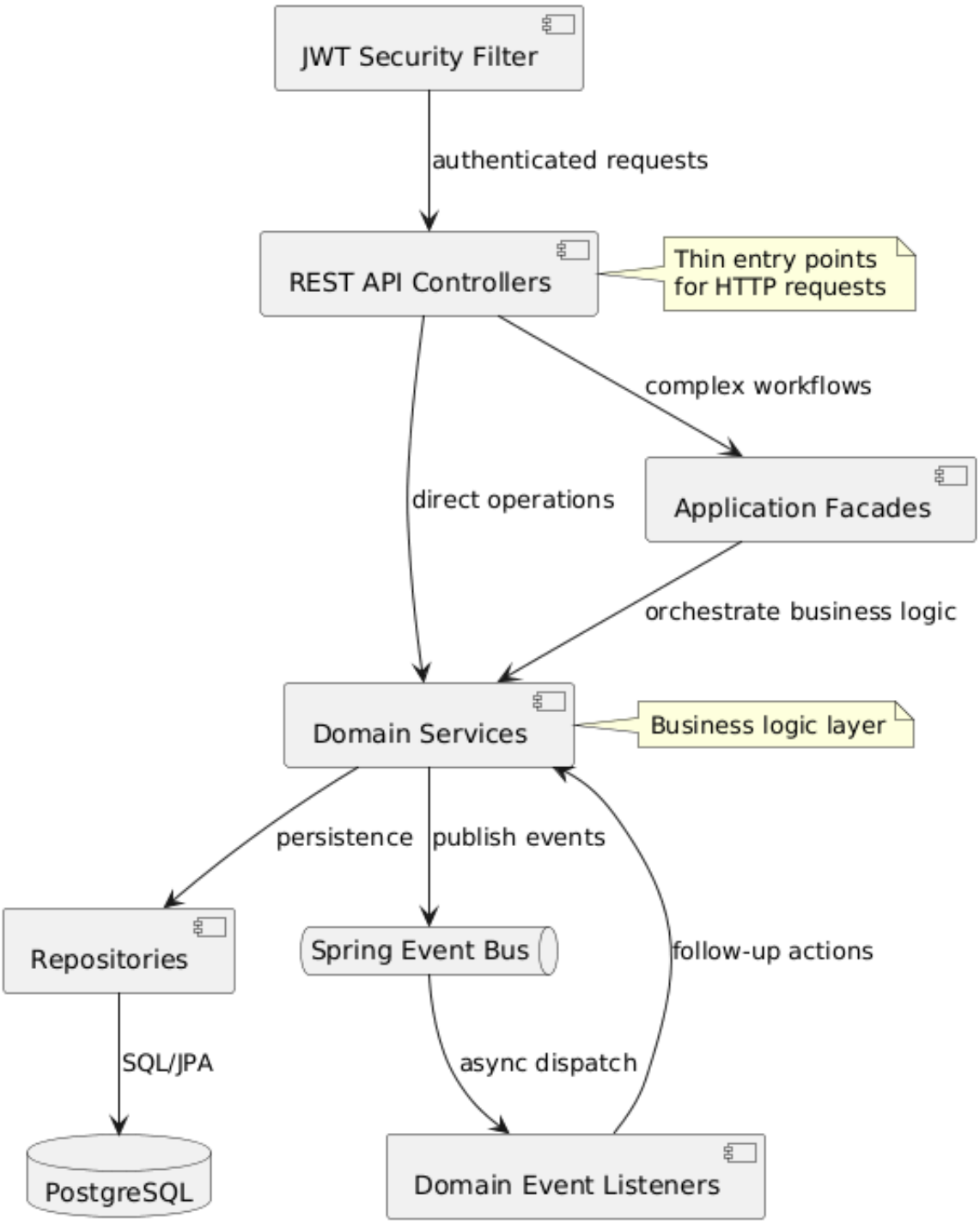
4. AsyncConfig cấu hình cơ chế asynchronous processing cho EventBus.
5. ExceptionHandler xử lý lỗi tập trung cho toàn bộ API.
6. JacksonConfig hỗ trợ serialization/deserialization dữ liệu JSON.

Guideline

Khi xây dựng Infrastructure / Technical Component View cần tuân thủ các nguyên tắc sau:

- Chỉ mô tả các technical component mang tính cross-cutting.
- Infrastructure layer cần độc lập với business domain.
- Các component cấu hình nên được gom theo nhóm chức năng.
- Security concern cần được tách riêng khỏi business logic.
- Các asynchronous infrastructure nên được biểu diễn rõ ràng.
- Exception handling nên được centralized.
- Không đưa business workflow vào sơ đồ infrastructure.
- Sơ đồ cần giúp người đọc hiểu được các thành phần kỹ thuật hỗ trợ toàn hệ thống.

4.2.5 Component & Connector (C&C) View



Hình 8: Component & Connector View của hệ thống

Sơ đồ Component & Connector (C&C) View mô tả các thành phần runtime của hệ thống và cách các thành phần này giao tiếp với nhau trong quá trình thực thi.

Khác với Component View tập trung vào cấu trúc thành phần, C&C View tập

trung vào:

- Runtime interaction.
- Luồng xử lý request.
- Event-driven communication.
- Quan hệ giao tiếp giữa các component.

Các thành phần chính trong sơ đồ bao gồm:

JWT Security Filter: Thực hiện xác thực và kiểm tra quyền truy cập trước khi request được xử lý bởi hệ thống.

REST API Controllers: Tiếp nhận request HTTP từ client và định tuyến request tới các facade hoặc service tương ứng.

Application Facades: Điều phối các workflow nghiệp vụ phức tạp giữa nhiều domain service khác nhau.

Domain Services: Xử lý business logic chính của hệ thống.

Repositories: Thực hiện persistence và truy vấn dữ liệu từ PostgreSQL.

Spring Event Bus: Cung cấp cơ chế giao tiếp bất đồng bộ giữa các thành phần thông qua event-driven architecture.

Domain Event Listeners: Lắng nghe và xử lý các event được publish từ domain services.

Luồng xử lý của C&C View

Luồng xử lý runtime của hệ thống được thực hiện như sau:

1. Request từ client được xác thực thông qua JWT Security Filter.
2. Request được chuyển tới REST API Controllers.
3. Controller:
    - Gọi trực tiếp domain services cho các thao tác đơn giản.
    - Gọi facade cho các workflow phức tạp cần orchestration.

4. Domain services xử lý business logic và thao tác với repositories để truy cập dữ liệu.
5. Repositories tương tác với PostgreSQL thông qua JPA/Hibernate.
6. Services có thể publish domain events tới Spring Event Bus.
7. EventBus thực hiện dispatch bất đồng bộ tới các listener.
8. Listener thực hiện các tác vụ follow-up hoặc kích hoạt xử lý tiếp theo.

Guideline cho C&C View

Khi xây dựng C&C View cần tuân thủ các nguyên tắc sau:

- C&C View phải tập trung vào runtime communication thay vì cấu trúc source code.
- Chỉ nên mô tả các runtime component quan trọng và các connector chính giữa chúng.
- Các connector cần thể hiện rõ:
  - synchronous communication.
  - asynchronous communication.
  - event-driven interaction.
- Các request flow chính cần được mô tả rõ ràng từ controller tới service và database.
- Các cơ chế cross-cutting như authentication, messaging hoặc event bus nên được thể hiện rõ trong runtime flow.
- Không nên đưa quá nhiều chi tiết implementation như entity, DTO hoặc class-level dependency vào C&C View.
- Các interaction bất đồng bộ nên được biểu diễn riêng biệt nhằm làm nổi bật kiến trúc event-driven.
- C&C View cần giúp người đọc nhanh chóng hiểu:

- Các runtime component chính của hệ thống.
- Các connector giữa các component.
- Luồng giao tiếp và xử lý request.
- Cơ chế giao tiếp đồng bộ và bất đồng bộ.

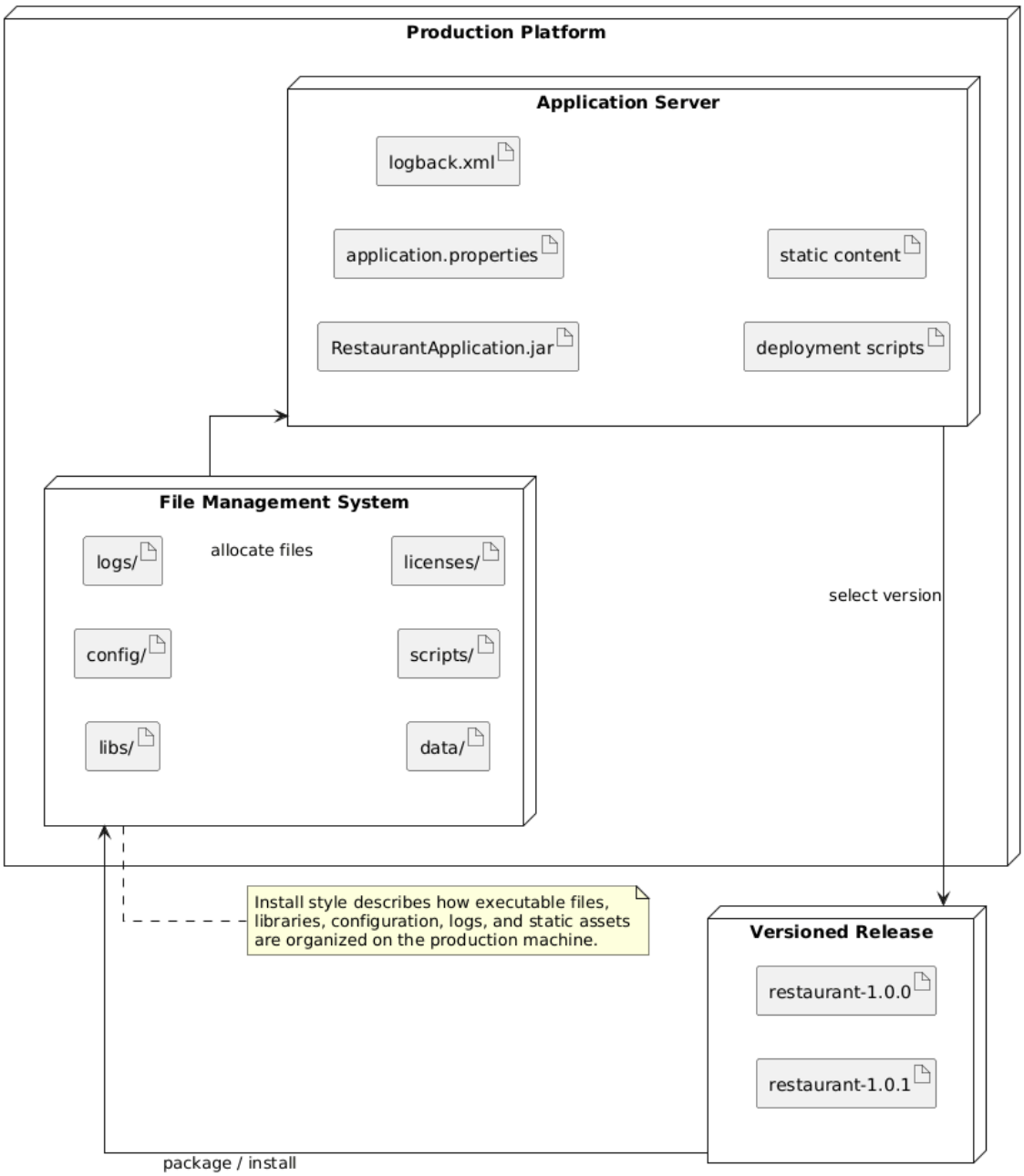
4.3 Allocation View

Allocation View mô tả cách hệ thống phần mềm được ánh xạ tới môi trường triển khai, cấu trúc cài đặt và tổ chức nhân sự phát triển hệ thống. View này giúp thể hiện mối quan hệ giữa kiến trúc phần mềm và môi trường thực tế trong quá trình triển khai, vận hành và phát triển hệ thống.

Trong hệ thống quản lý nhà hàng, Allocation View được mô tả thông qua ba góc nhìn chính:

- Install Style
- Deployment Style
- Work Assignment Style

4.3.1 Install Style



Hình 9: Install Style của hệ thống

Sơ đồ Install Style mô tả cách các thành phần phần mềm, file cấu hình và tài nguyên hệ thống được tổ chức trên môi trường production.

Hệ thống được triển khai trên một *Production Platform* bao gồm hai thành phần chính:

Application Server: Chứa các artifact cần thiết để chạy hệ thống:

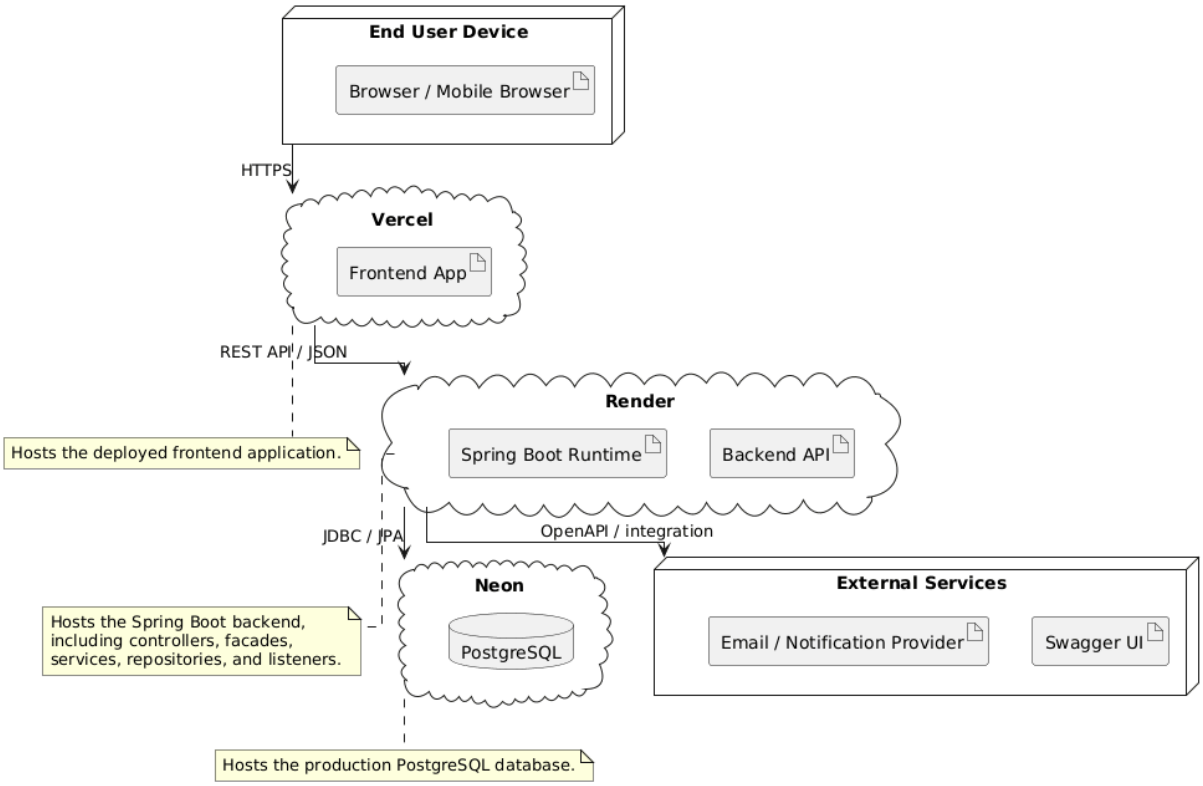
- *RestaurantApplication.jar*: file thực thi chính của ứng dụng Spring Boot.
- *application.properties*: file cấu hình hệ thống.
- *logback.xml*: cấu hình logging.
- *deployment scripts*: script phục vụ triển khai hệ thống.
- *static content*: tài nguyên tĩnh như HTML, CSS, JavaScript và hình ảnh.

File Management System: Quản lý cấu trúc thư mục và các file cài đặt của hệ thống:

- *libs/*: chứa thư viện phụ thuộc.
- *config/*: chứa file cấu hình.
- *logs/*: chứa log runtime.
- *data/*: chứa dữ liệu hệ thống.
- *scripts/*: chứa script triển khai và vận hành.
- *licenses/*: chứa thông tin giấy phép phần mềm.

Ngoài ra, hệ thống còn hỗ trợ quản lý nhiều phiên bản phát hành thông qua các release như: *restaurant-1.0.0* và *restaurant-1.0.1*. Các phiên bản này được package và cài đặt vào môi trường production thông qua File Management System.

4.3.2 Deployment Style



Hình 10: Deployment Style của hệ thống

Sơ đồ Deployment Style mô tả cách hệ thống được triển khai trên các node và nền tảng hạ tầng thực tế.

End User Device: Người dùng truy cập hệ thống thông qua trình duyệt web hoặc mobile browser.

Vercel: Được sử dụng để triển khai frontend application của hệ thống. Frontend chịu trách nhiệm hiển thị giao diện người dùng và gửi request tới backend thông qua REST API.

Render: Được sử dụng để triển khai backend API chạy trên nền tảng Spring Boot Runtime. Backend bao gồm:

- Controllers
- Facades
- Services
- Repositories

- Event listeners

Neon: Được sử dụng để triển khai cơ sở dữ liệu PostgreSQL trên cloud nhằm lưu trữ dữ liệu nghiệp vụ của hệ thống.

External Services: Bao gồm các dịch vụ tích hợp bên ngoài như:

- Swagger UI phục vụ kiểm thử và tài liệu hóa API.
- Email / Notification Provider phục vụ gửi thông báo hệ thống.

Luồng triển khai: Người dùng truy cập frontend trên Vercel thông qua HTTPS, frontend gửi REST API request tới backend trên Render, sau đó backend tương tác với PostgreSQL trên Neon thông qua JDBC/JPA.

4.3.3 Work Assignment Style

Work Assignment Style mô tả việc phân bổ trách nhiệm phát triển các module và nhiệm vụ trong hệ thống cho các nhóm phát triển tương ứng.

Hệ thống được phát triển thông qua nhiều nhóm khác nhau bao gồm:

- Backend Team
- Frontend Team
- QA Team
- DevOps Team

Các nhiệm vụ chính của dự án bao gồm:

- Requirements Analysis
- Architecture Design
- Auth Module
- Menu Module
- Order Module
- Kitchen Module
- Inventory Module

- Frontend Dashboard
- Integration Testing
- Deployment & Monitoring

Mỗi task được phân công:

- Team chịu trách nhiệm chính.
- Team hỗ trợ.
- Thời gian thực hiện.
- Module phụ thuộc.
- Deliverables đầu ra.
- Trạng thái thực hiện.
- Assignee chịu trách nhiệm trực tiếp.

Ngoài ra, Work Assignment Style còn sử dụng Gantt Chart để mô tả timeline phát triển hệ thống từ tuần 1 đến tuần 8. Gantt Chart giúp thể hiện:

- Trình tự thực hiện các task.
- Quan hệ phụ thuộc giữa các module.
- Phân bổ thời gian cho từng nhóm phát triển.
- Tiến độ tổng thể của dự án.

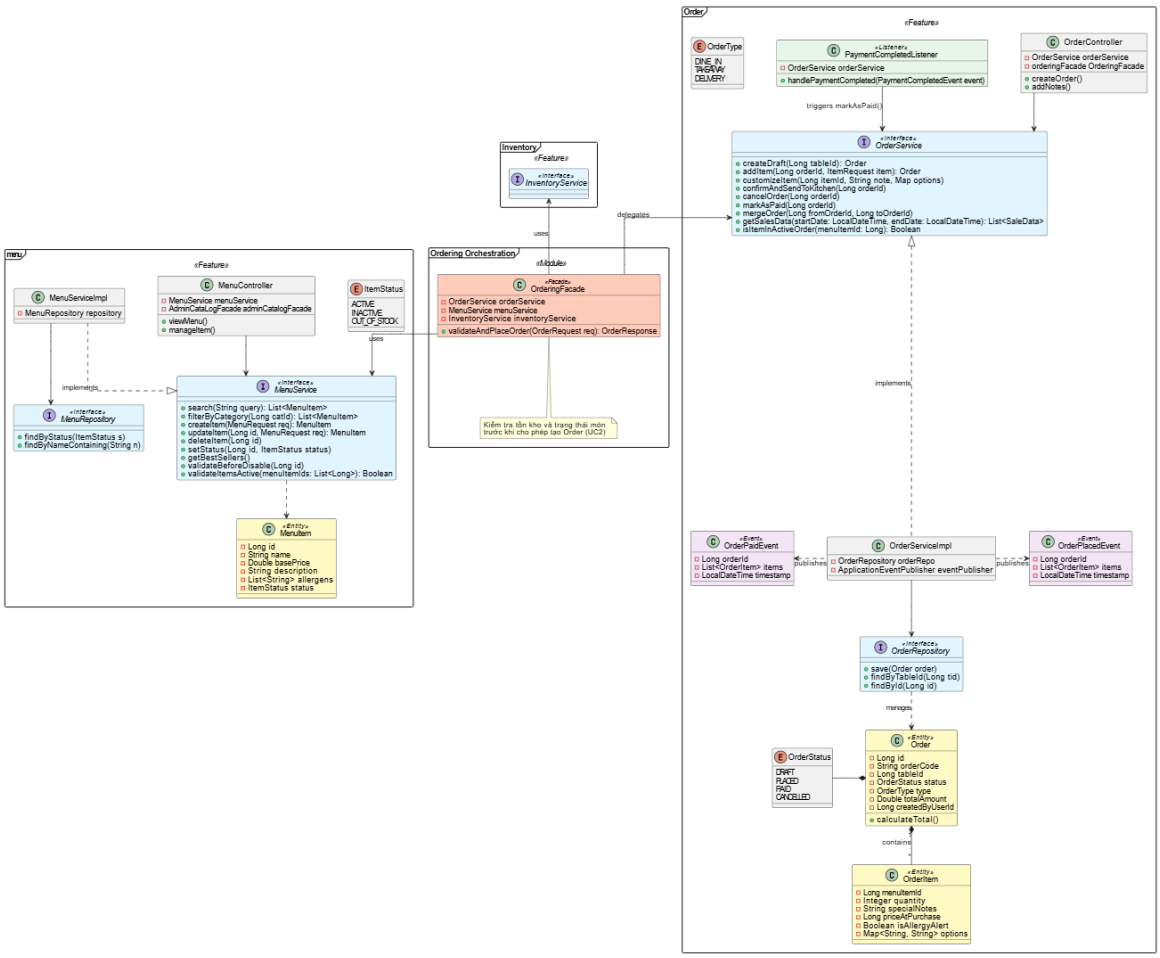
Thông qua Work Assignment Style, nhóm phát triển có thể dễ dàng quản lý tài nguyên, phân chia trách nhiệm và theo dõi tiến độ thực hiện của toàn bộ hệ thống.

5 Thiết kế class diagram

Do thiết kế class diagram quá lớn không thể trình bày trên một trang tài liệu. Vì thế, tài liệu này phân tích thiết kế của class diagram thành những tập hợp các thành phần nhỏ của một số module có liên quan đến nhau cho luồng thực thi nghiệp vụ nào đó.

Thiết kế đầy đủ: Thiết kế Class Diagram

5.1 Nghiệp vụ 1: Nhân viên thực hiện chọn món và tạo order



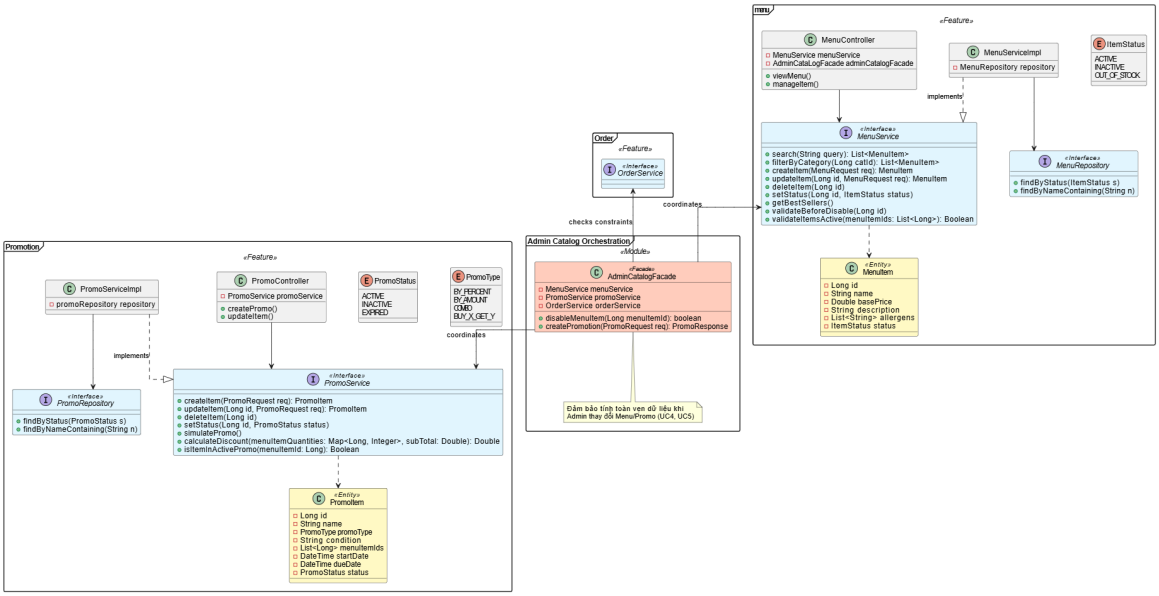
Hình 11: Thiết kế class diagram cho nghiệp vụ tạo order

Khi thực thi các module trong hình sẽ có những tương tác phụ thuộc lẫn nhau như trong hình 1 với các API như sau:

- `createDraft(Long tableId): Order`: trả về một order nháp được khởi tạo. Được gọi trong `createOrder` thuộc class `OrderController`.
- `addItem(Long orderId ItemRequest item): Order`: thực thi khi nhân viên thêm các món ăn vào một order vừa mới tạo.
- `customizeItem(Long itemId, String note, Map options)`: thực thi tạo một OrderItem bao gồm **note** (Lưu ý của khách hàng khi order), **options** (Những yêu cầu bổ sung như topping hay thêm cơm)

Bên cạnh đó, việc thực hiện logic kiểm tra và thực sự tạo một order trong hệ thống sẽ được kiểm tra và quyết định tạo order bởi module **OrderFacade** khác. Thiết kế này đảm bảo nguyên lý Single Responsibility trong SOLID, khi module Order không cần biết việc kiểm tra menuItem hợp lệ diễn ra thế nào. Ngoài ra, việc đặt code được hiện thực ra một module khác như vậy giúp dễ bảo trì logic kiểm tra trạng thái của một thực thể thuộc module khác.

5.2 Nghiệp vụ 2: Quản trị viên thiết lập Catalog (Menu và Khuyến mãi)



Hình 12: Thiết kế class diagram cho nghiệp vụ quản lý danh mục và khuyến mãi

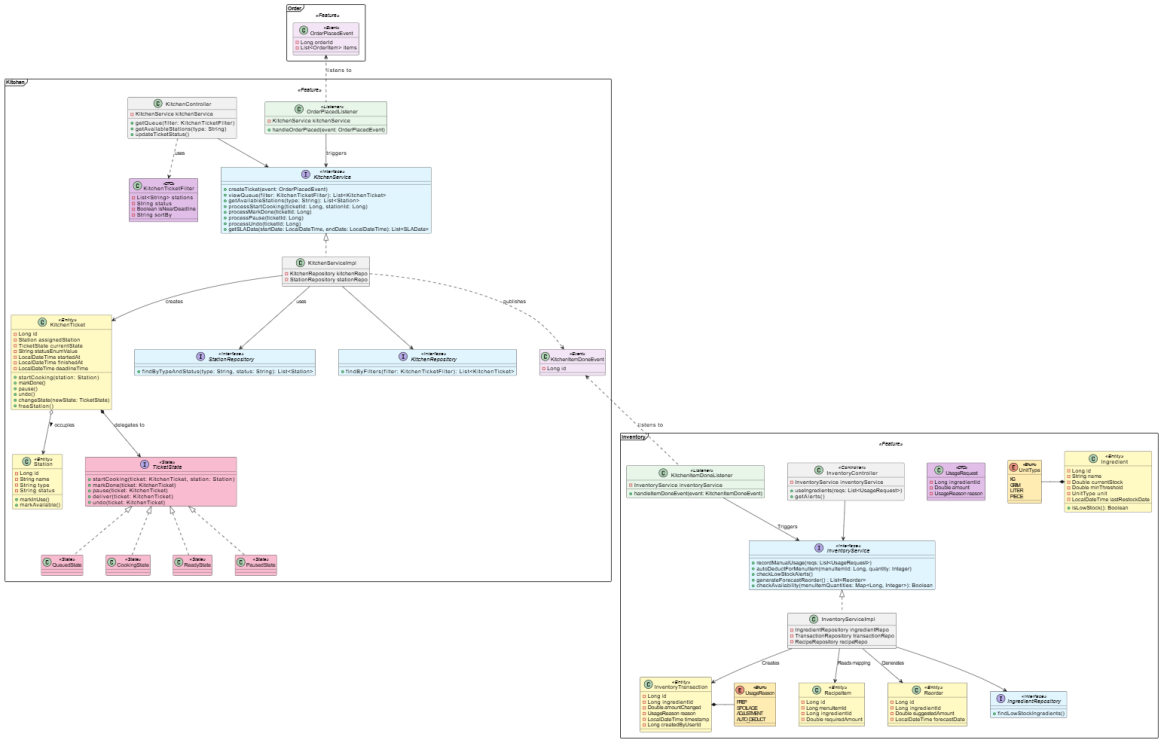
Nghiệp vụ quản lý thực đơn và các chương trình khuyến mãi đòi hỏi tính toàn vẹn dữ liệu cao giữa các phân hệ. Các API chính tham gia vào tiến trình này bao gồm:

- `createItem(MenuRequest req): MenuItem` và `createItem(PromoRequest req): PromoItem`: tạo mới các thực thể món ăn hoặc chương trình khuyến mãi thông qua các Service tương ứng.
- `setStatus(Long id, ItemStatus status)`: thực hiện cập nhật trạng thái hoạt động của thực đơn (Active, Inactive, Out of Stock).
- `disableMenuItem(Long menuItemId): boolean`: API được bọc bởi **AdminCatalogFacade** để thực hiện vô hiệu hóa một món ăn một cách an toàn.

Để tránh việc dữ liệu bị lỗi (ví dụ: xóa một món ăn đang nằm trong một chương trình khuyến mãi đang chạy hoặc một order chưa thanh toán), hệ thống áp dụng **Facade Pattern** thông qua `AdminCatalogFacade`. Module này đóng vai trò điều phối, sẽ thực hiện gọi chéo sang `OrderService` và `PromoService` để xác thực các ràng buộc nghiệp

vụ trước khi ra lệnh cho `MenuService` cập nhật cơ sở dữ liệu.

5.3 Nghiệp vụ 3: Xử lý nhà bếp và Tự động trừ tồn kho



Hình 13: Thiết kế class diagram cho nghiệp vụ chế biến và quản lý nguyên liệu

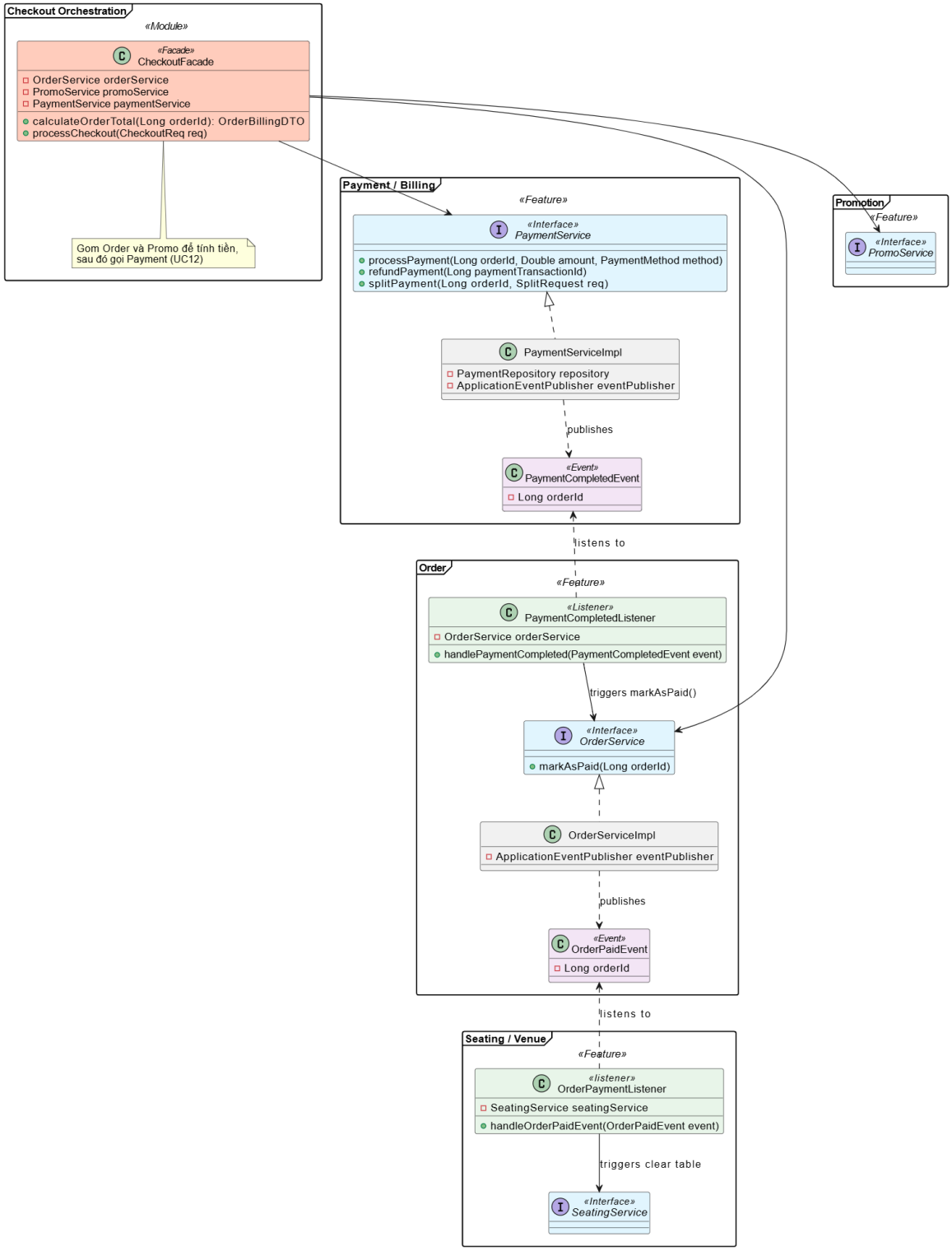
Đây là luồng nghiệp vụ hoạt động chủ yếu ngầm ở phía sau (Back of House), phụ thuộc mạnh vào cơ chế giao tiếp bất đồng bộ thông qua Event. Các API và hành vi chính bao gồm:

- `handleOrderPlaced(event: OrderPlacedEvent)`: Listener phía Kitchen nhận tín hiệu từ luồng Order, sau đó gọi `createTicket` để tạo danh sách công việc cho đầu bếp.
- `changeState(newState: TicketState)`: quản lý vòng đời của một phiếu nấu ăn thông qua **State Pattern** (từ `QueuedState` sang `CookingState` và cuối cùng là `ReadyState`).
- `processMarkDone(ticketId: Long)`: Đầu bếp xác nhận hoàn thành món ăn. Hàm này sẽ phát ra sự kiện `KitchenItemDoneEvent`.
- `autoDeductForMenuItem(menuItemId: Long, quantity: Integer)`: Listener phía Inventory nhận sự kiện hoàn thành món, tra cứu `RecipeItem` để

trừ nguyên liệu thô tự động.

Sự liên kết lỏng lẻo (loose coupling) ở đây mang lại lợi ích: các thao tác của nhà bếp hoàn toàn độc lập với kho. Sự tách biệt module và sử dụng **Observer Pattern** để giao tiếp giúp code dễ maintain theo từng module cũng như giảm sự phụ thuộc trong luồng thực thi giữa cả 2 module.

5.4 Nghiệp vụ 4: Thanh toán hóa đơn và Quản lý trạng thái bàn



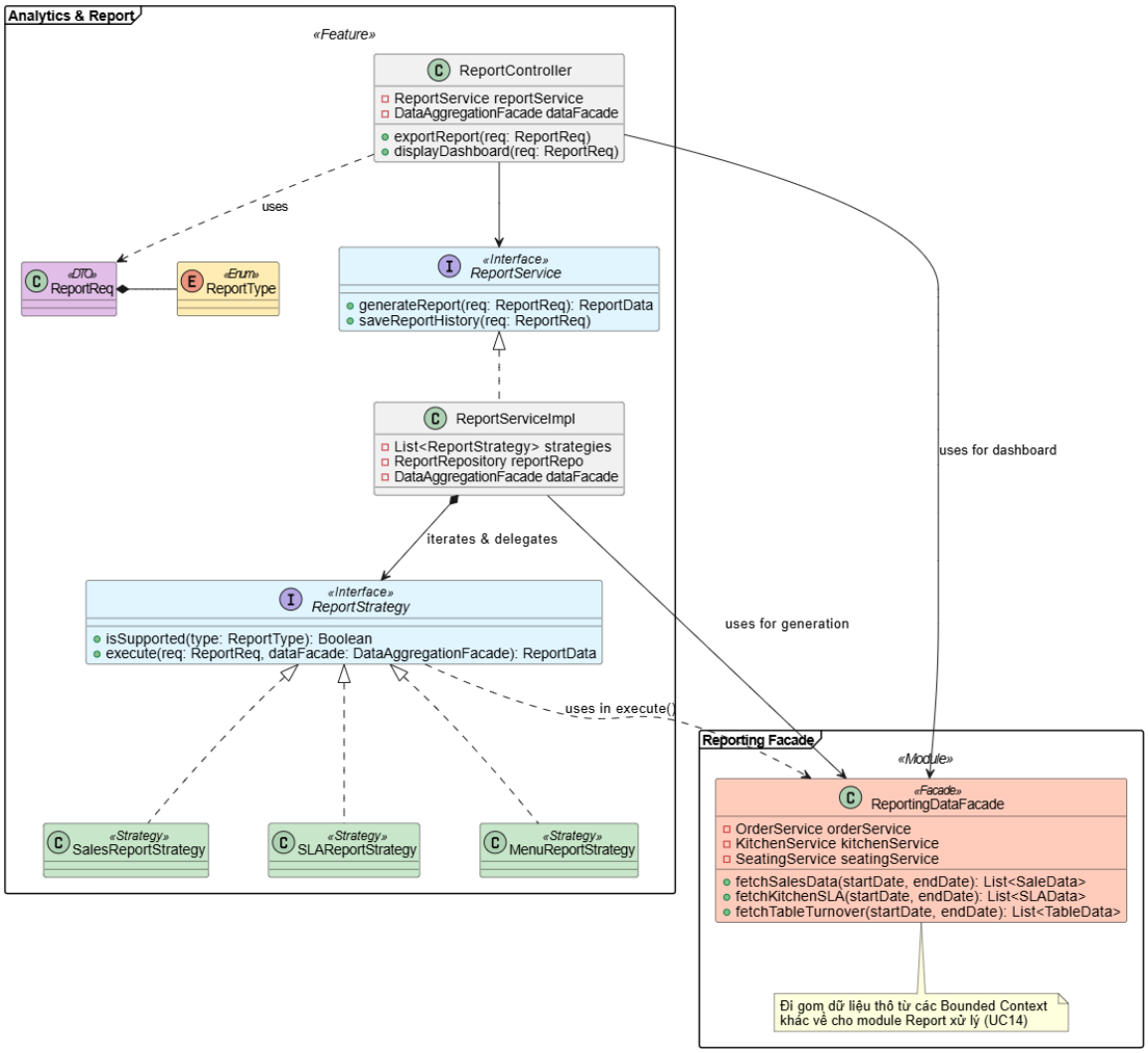
Hình 14: Thiết kế class diagram cho nghiệp vụ thanh toán và giải phóng bàn

Giai đoạn cuối cùng của chu trình phục vụ khách hàng bao gồm việc tính toán chi phí, xử lý thanh toán và cập nhật sơ đồ nhà hàng.

- `calculateOrderTotal(Long orderId)`: được gọi qua `CheckoutFacade` nhằm gom dữ liệu từ `OrderService` và tính toán mức giảm giá thông qua `PromoService`.
- `processPayment(Long orderId, Double amount, PaymentMethod method)`: thực thi giao dịch thanh toán và phát ra `PaymentCompletedEvent` khi thành công.
- `markAsPaid(Long orderId)`: `OrderService` lắng nghe giao dịch thành công để cập nhật trạng thái hóa đơn và tiếp tục kích hoạt `OrderPaidEvent`.
- `markAsDirty()`: API thuộc `Table` entity, được trigger bởi `OrderPaymentListener` thuộc module Seating để tự động báo cho nhân viên dọn bàn.

Việc áp dụng `CheckoutFacade` giúp hệ thống giấu đi sự phức tạp của việc tính toán giá trị hóa đơn (bao gồm thuế, tip, khuyến mãi chồng chéo). Đồng thời, phản ứng dây chuyền qua Event (Payment → Order → Seating) đảm bảo sơ đồ nhà hàng (Floor plan) giúp tạo ra giao tiếp tự động giữa các module.

5.5 Nghiệp vụ 5: Phân tích dữ liệu và Xuất báo cáo



Hình 15: Thiết kế class diagram cho nghiệp vụ phân tích và tổng hợp báo cáo

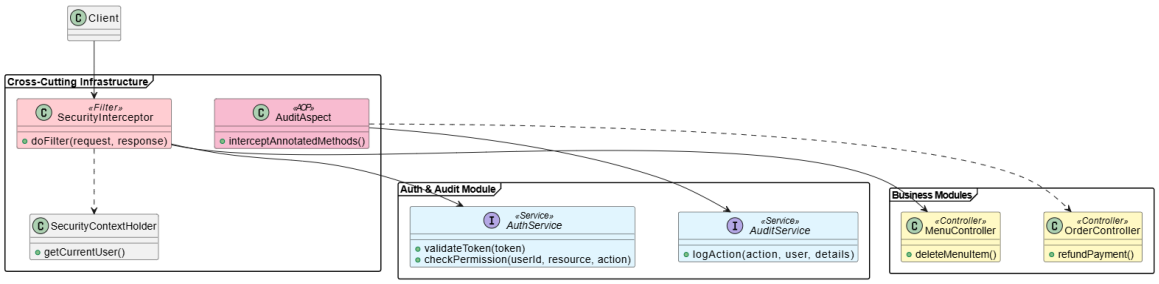
Module Analytics hoạt động độc lập và không can thiệp vào tiến trình giao dịch của các phân hệ khác (Non-blocking).

- `generateReport(req: ReportReq): ReportData`: điểm vào duy nhất cho mọi yêu cầu báo cáo.
- `execute(req, dataFacade)`: phương thức thuộc giao diện `ReportStrategy`. Tùy thuộc vào loại báo cáo, hệ thống sẽ gọi `SalesReportStrategy`, `SLAReportStrategy` hoặc `MenuReportStrategy`.
- `fetchSalesData`, `fetchKitchenSLA`: Các API gom dữ liệu thô từ các module

nghiệp vụ khác thông qua `ReportingDataFacade`.

Thiết kế kết hợp giữa **Strategy Pattern** và **Facade Pattern** giúp hệ thống Report dễ dàng mở rộng. Khi nhà hàng cần thêm một loại báo cáo mới (ví dụ: Báo cáo nhân sự), ta chỉ cần tạo thêm một Strategy class mới mà không làm ảnh hưởng đến mã nguồn cũ (tuân thủ nguyên lý Open/Closed trong SOLID).

5.6 Nghiệp vụ 6: Kiểm soát truy cập và Lưu vết hệ thống (Cross-Cutting)



Hình 16: Thiết kế class diagram cho hạ tầng phân quyền và Audit

Để đảm bảo tính bảo mật và truy vết (đặc biệt trong các thao tác nhạy cảm như hoàn tiền, xóa món), hệ thống áp dụng kiến trúc Cross-Cutting Concerns.

- `doFilter(request, response)`: Thuộc `SecurityInterceptor`, chặn mọi luồng API đi vào hệ thống để xác thực Token và kiểm tra vai trò người dùng qua `AuthService`.
- `interceptAnnotatedMethods()`: Thuộc `AuditAspect`, tự động bao bọc các Controller nhạy cảm.
- `logAction(action, user, details)`: Ghi nhận lịch sử thao tác vào cơ sở dữ liệu.

Việc sử dụng **AOP (Aspect-Oriented Programming)** thông qua `AuditAspect` là một thiết kế tối ưu. Nó giúp bóc tách hoàn toàn logic kiểm tra quyền và ghi log ra khỏi logic nghiệp vụ chính (Business Logic) bên trong các Service. Nhờ đó, mã nguồn của `OrderController` hay `MenuController` giữ được sự gọn gàng và dễ đọc.

6 Link mã nguồn

Github : https://github.com/nguyen2966/Restaurant_Project

- Mô tả mã nguồn được cung cấp trong file **README.md**
- Test plan được trình bày trong **TestPlan.md**